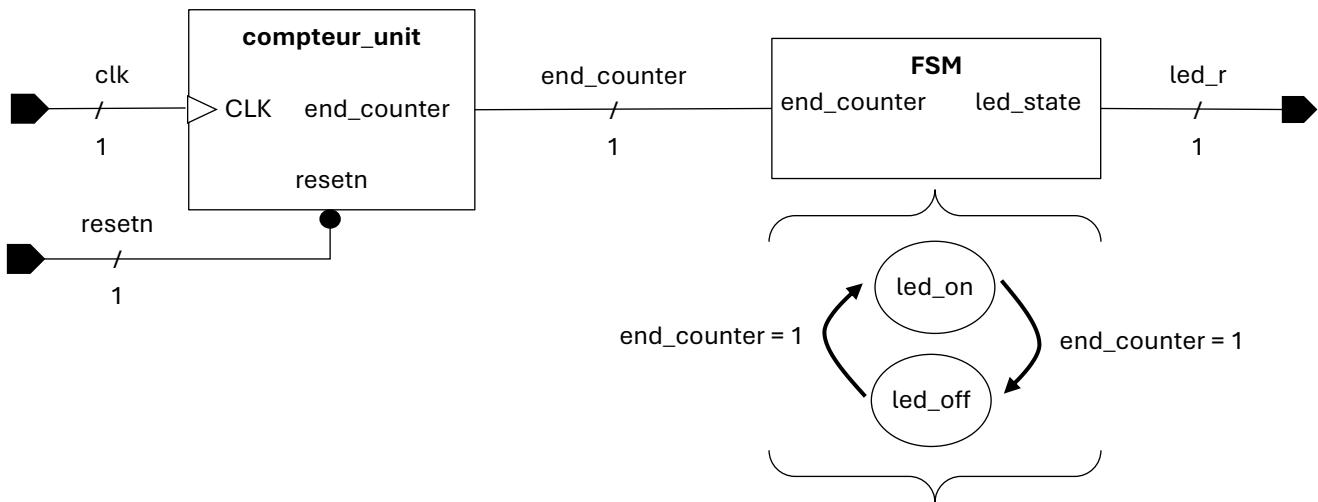


TP4 – Pilotage de LED et mémoire

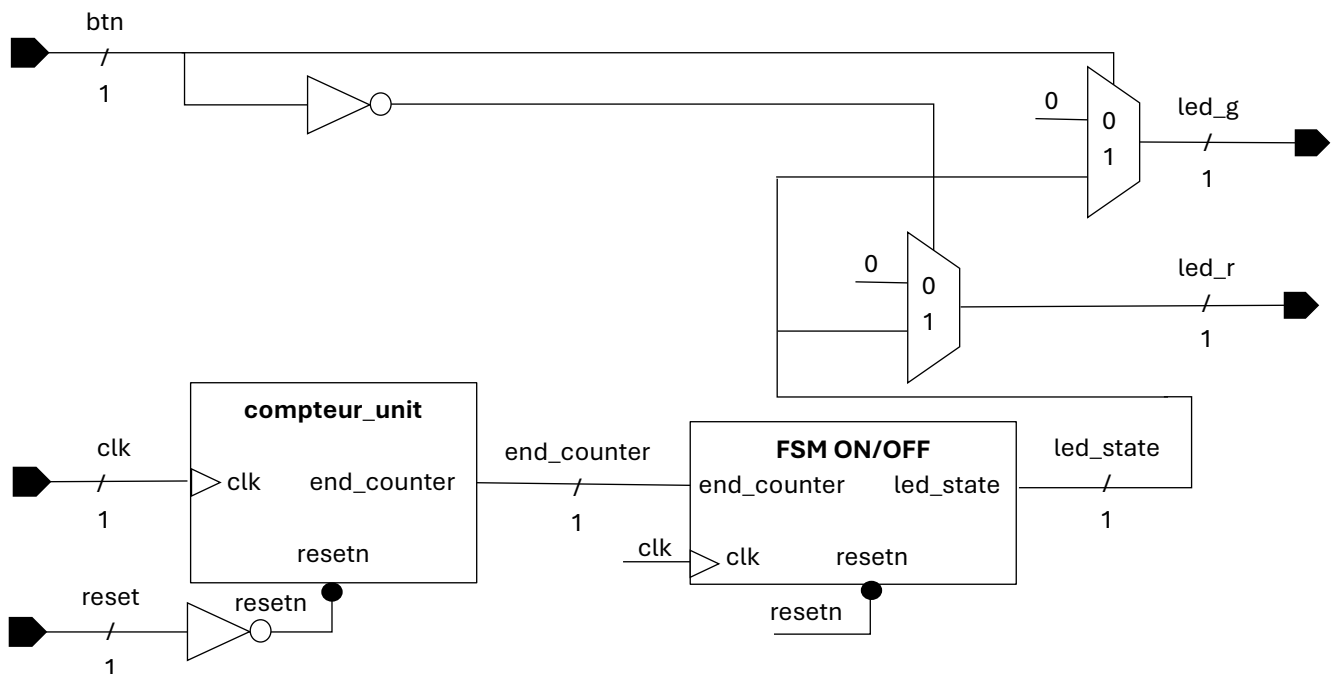
1. Créez une architecture RTL permettant de faire clignoter une LED (par exemple la led0_r) en utilisant le module Counter_unit du TP2 et une machine à état.



Le compteur_unit produit une sortie end_counter qui passe à 1 sur une période de clk lorsque la fin de comptage est atteinte.

Pour pouvoir piloter le clignotement d'une LED, on peut utiliser une machine d'état avec deux états ON et OFF. Le changement d'état se déclenchera sur end_counter.

2. Modifiez votre architecture pour piloter une LED rouge et une LED verte. Lorsque le bouton_0 est appuyé, la LED verte est allumée, sinon la LED rouge est allumée.



Pour piloter la LED en rouge ou en vert on utilise des multiplexeurs

led_g = led_drive quand btn_0 = 1 sinon 0

led_r = led_drive quand btn_0 = 0 sinon 0

3. Rédigez le code VHDL de votre architecture.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tp_led_driver is
    generic (
        cible : integer := 20
    );
    port (
        clk      : in std_logic;
        reset    : in std_logic;
        btn      : in std_logic;
        led_r    : out std_logic;
        led_g    : out std_logic
    );
end tp_led_driver;

architecture behavioral of tp_led_driver is

    type state is (led_on, led_off);
    signal current_state : state;
    signal next_state : state;
    signal resetn : std_logic := '0';
    signal end_counter : std_logic := '0';
    signal led_state : std_logic := '0';

begin
    cpt: entity work.counter_unit(behavioral)
        generic map (
            cible => cible
        )
        port map (
            clk => clk,
            resetn => resetn,
            end_counter => end_counter
        );

    process(clk, resetn)
    begin
        if resetn = '0' then
            current_state <= led_off;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

    process(current_state, end_counter)
    begin
        next_state <= current_state;
        case current_state is
            when led_off =>
                if end_counter = '1' then
                    next_state <= led_on;
                end if;
            when led_on =>
                if end_counter = '1' then
                    next_state <= led_off;
                end if;
        end case;
    end process;

    resetn <= not reset;
    led_state <= '1' when current_state = led_on else '0';
    led_r <= led_state when btn = '0' else '0';
    led_g <= led_state when btn = '1' else '0';

end behavioral;
```

Appel du compteur de cycle
counter_unit

Process séquentiels de
contrôle de la machine
d'état

Process combinatoire
de contrôle de la LED

4. Réalisez une simulation en rédigeant un test bench.

- Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn, btn

Liste des signaux de sortie à vérifier : led_r, led_g

Liste des signaux internes à vérifier : end_counter, led_state

Listes des états à vérifier: current_state, next_state

L'horloge clk est définie à 100Mhz

Note : les simulations ont été faite avec une cible de 20 soit un temps de comptage du compteur unitaire de 200ns

- Déroulé de la validation et du test bench

1. Test du reset au démarrage

1.1. Les entrées seront définies comme suit :

- **resetn = 0**
- btn = 0

1.2. Les sorties seront vérifiées

- led_r = 0 , led_g= 0

1.3. Les signaux : états internes à vérifier

- led_state = 0 resetté
- end_counter = 0 : counter_unit resetté
- current_state = led_off : au reset l'état de départ est led_off

1.4. Sur le test bench

- on teste les 2 sorties de led : led_r=0, led_g= 0 après $(2*(2*cible)+cible)*periode(clk)+1ns$
- on attend $cible*period + 1ns$ pour lancer la suite

2. Test du démarrage après reset

2.1. Les entrées seront définies comme suit :

- **resetn = 1**
- btn = 0

2.2. Les sorties seront vérifiées

- **led_r = 0/1** à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)
- led_g= 0

2.3. Les signaux : états internes à vérifier

- led_state = 0/1 à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)
- end_counter passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)
- current_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)

2.4. Sur le test bench

- on teste les 2 sorties de led : led_r=1, led_g= 0 après $cible*periode(clk)+1ns$
- on attend $4*2*cible*period + 1ns$ pour lancer la suite

3. Test de l'appui sur le bouton

3.1. Les entrées seront définies comme suit :

- resetn =1
- **btn = 1**

3.2. Les sorties seront vérifiées

- led_r = 0 ,
- **led_g= 0/1** à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)

3.3. Les signaux : états internes à vérifier

- led_state = 0/1 à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)
- end_counter passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)

- `current_state = led_off/led_on` à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
- 3.4. Sur le test bench
- on teste les 2 sorties de led : `led_r = 0`, `led_g = 1` après `1*periode(clk)+1ns`
 - on attend `4*2*cible*period + 1ns` pour lancer la suite
4. Le bouton est relâché
- 4.1. Les entrées seront définies comme suit :
- `resetrn = 1`
 - **`btn = 0`**
- 4.2. Les sorties seront vérifiées
- `led_g = 0`,
 - **`led_r = 0/1`** à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
- 4.3. Les signaux : états internes à vérifier
- `led_state = 0/1` à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
 - `end_counter` passe à 1 sur une période de `clk` quand le compteur_unit compte (`cible - 1`)
 - `current_state = led_off/led_on` à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
- 4.4. Sur le test bench
- on teste les 2 sorties de led : `led_r = 1`, `led_g = 0` après `1*periode(clk)+1ns`
 - on attend `4*2*cible*period + 1ns` pour terminer le test

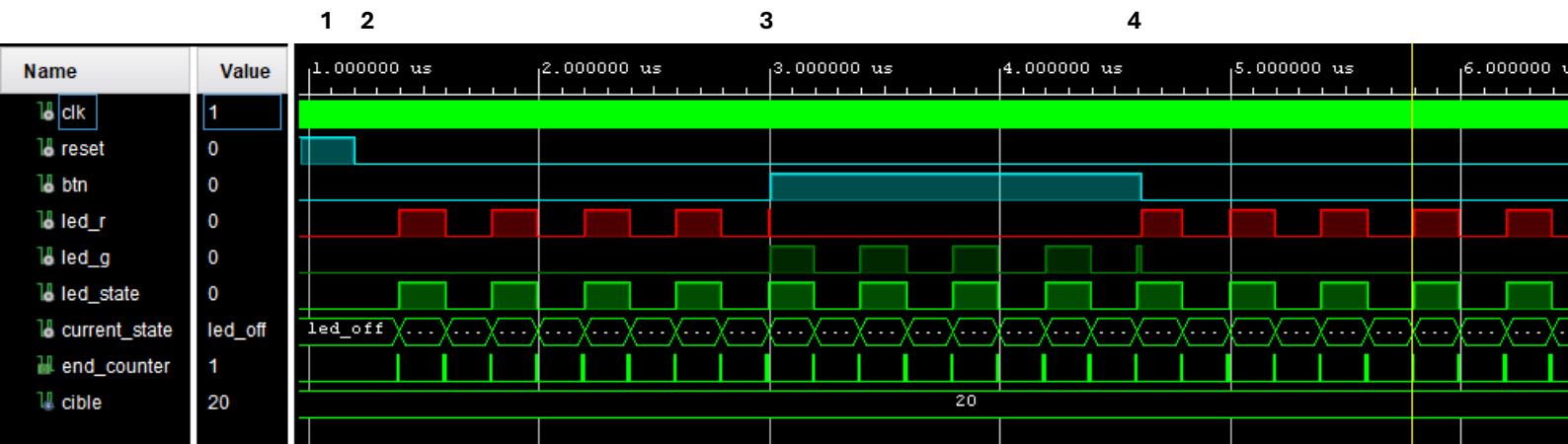
- Test bench

Le fichier de test bench est `tb_tp_led_driver.vhd`

- Résultat des test automatiques du test_bench

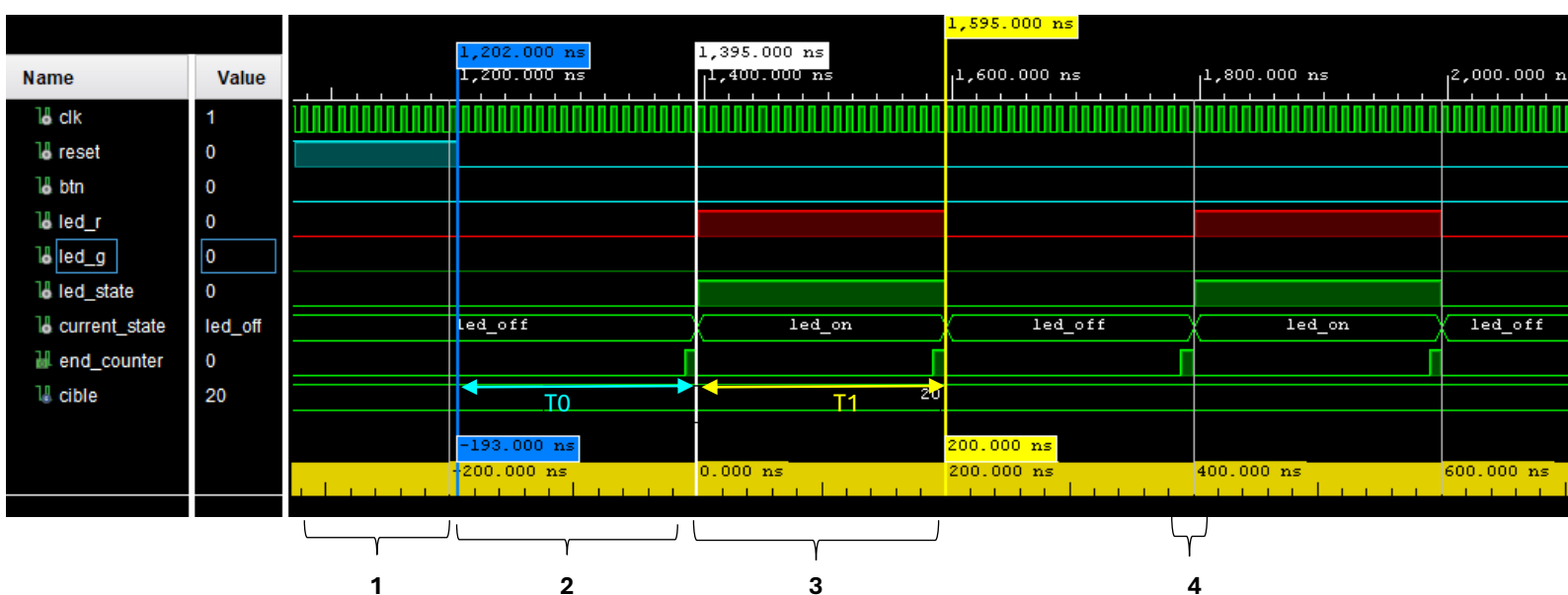
```
# run 8000ns
Note: PASS : TEST reset : la led est eteinte OK
Time: 1001 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
Note: PASS : TEST demarrage après reset avec btn non appuyé : led en rouge OK
Time: 1403 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
Note: PASS : TEST btn appuyé : led en vert OK
Time: 3015 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
Note: PASS : TEST btn relâché : led en rouge OK
Time: 4627 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
Note: Nombre de test FAIL : 0AUCUN TEST FAILED
Time: 6228 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_tp_led_driver_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 8000ns
] launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:07 . Memory (MB): peak = 2925.070 ; gain = 5.371
```

- Validation fonctionnelle



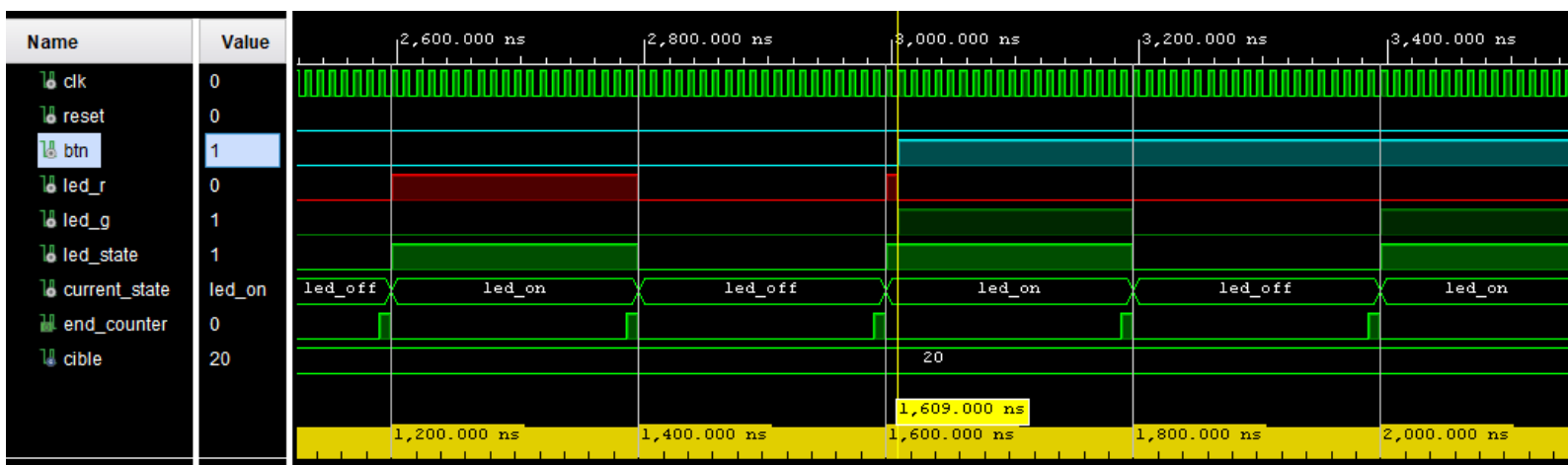
- 1- Test du reset: la LED est éteinte ($\text{led_r} = 0$ et $\text{led_g} = 0$)
le compteur_unit est resetté ($\text{end_counter} = 0$)
la FSM est dans l'état led_off
- 2- Démarrage après reset: le counter_unit démarre
la FSM bascule de led_on à led_off à chaque fin de cycle de comptage ($\text{end_counter} = 1$)
La LED est en rouge car le bouton n'est pas appuyé ($\text{btn} = 0$), elle clignote selon le cycle de la FSM
- 3- Appui sur le bouton ($\text{btn} = 1$)
le counter_unit continue de compter
la FSM continue de basculer de led_on à led_off
La LED est en vert car le bouton est appuyé ($\text{btn} = 1$), elle clignote selon le cycle de la FSM
- 4- On relâche le bouton ($\text{btn} = 0$)
le counter_unit continue de compter
la FSM continue de basculer de led_on à led_off
La LED est en rouge car le bouton est relâché ($\text{btn} = 0$), elle clignote selon le cycle de la FSM

- Validation du démarrage après reset



- 1- Quand reset est à 1: tous les registres sont resettés $\text{end_counter} = 0$, la fsm est dans l'état led_off, la LED est éteinte
- 2- Quand reset passe à 0 : le compteur unitaire s'incrémente sur les fronts montants d'horloge, le premier cycle de comptage n'est pas complet car au redémarrage $\text{d_counter} = 1$ en raison de la logique combinatoire de d_counter donc q_counter reçoit 1 au lieu de zéro sur le premier front d'horloge.
le premier cycle de comptage dure donc $T_0 = 193\text{ns}$ au lieu de $T_1 = 200\text{ns}$ comme pour le deuxième cycle
- 3- la led s'allume en rouge après le premier cycle unitaire ($\text{led_r} = 1$)
- 4- la machine d'état change d'état un coup d'horloge après la détection du compteur ($\text{end_counter} = 1$)

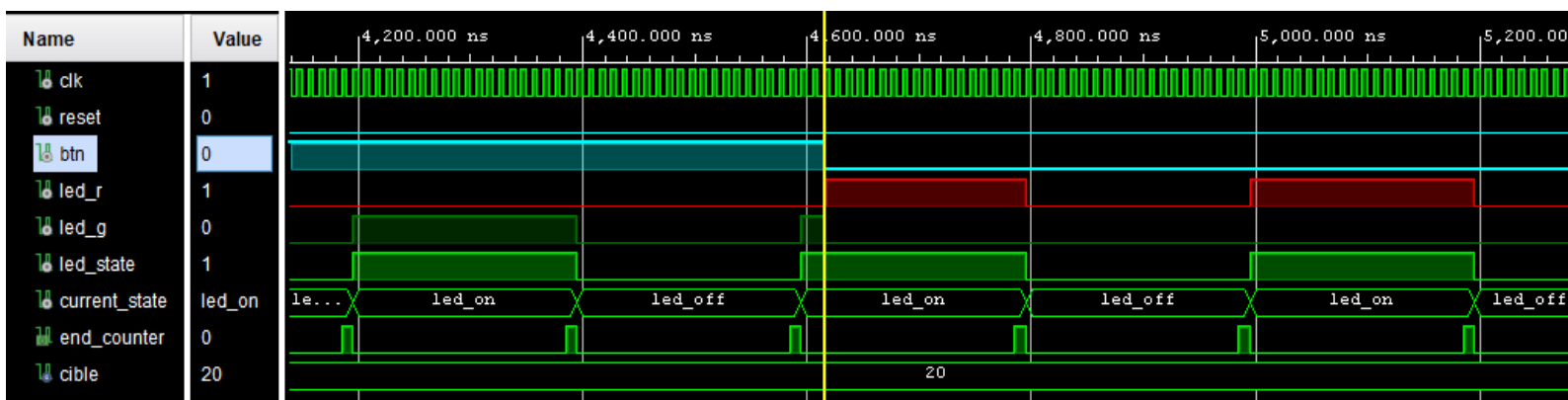
- Validation de l'appui sur le bouton btn



Quand le bouton est appuyé, la LED passe immédiatement en vert de façon asynchrone. sur un même cycle d'allumage la LED sera rouge puis verte

Le cycle de comptage et la FSM continuent sans être impacté par l'appui sur le bouton

- Validation du bouton relâché



Quand le bouton est relâché, la LED passe immédiatement en rouge de façon asynchrone. sur un même cycle d'allumage la LED sera verte puis rouge

Le cycle de comptage et la FSM continuent sans être impacté par l'état du bouton

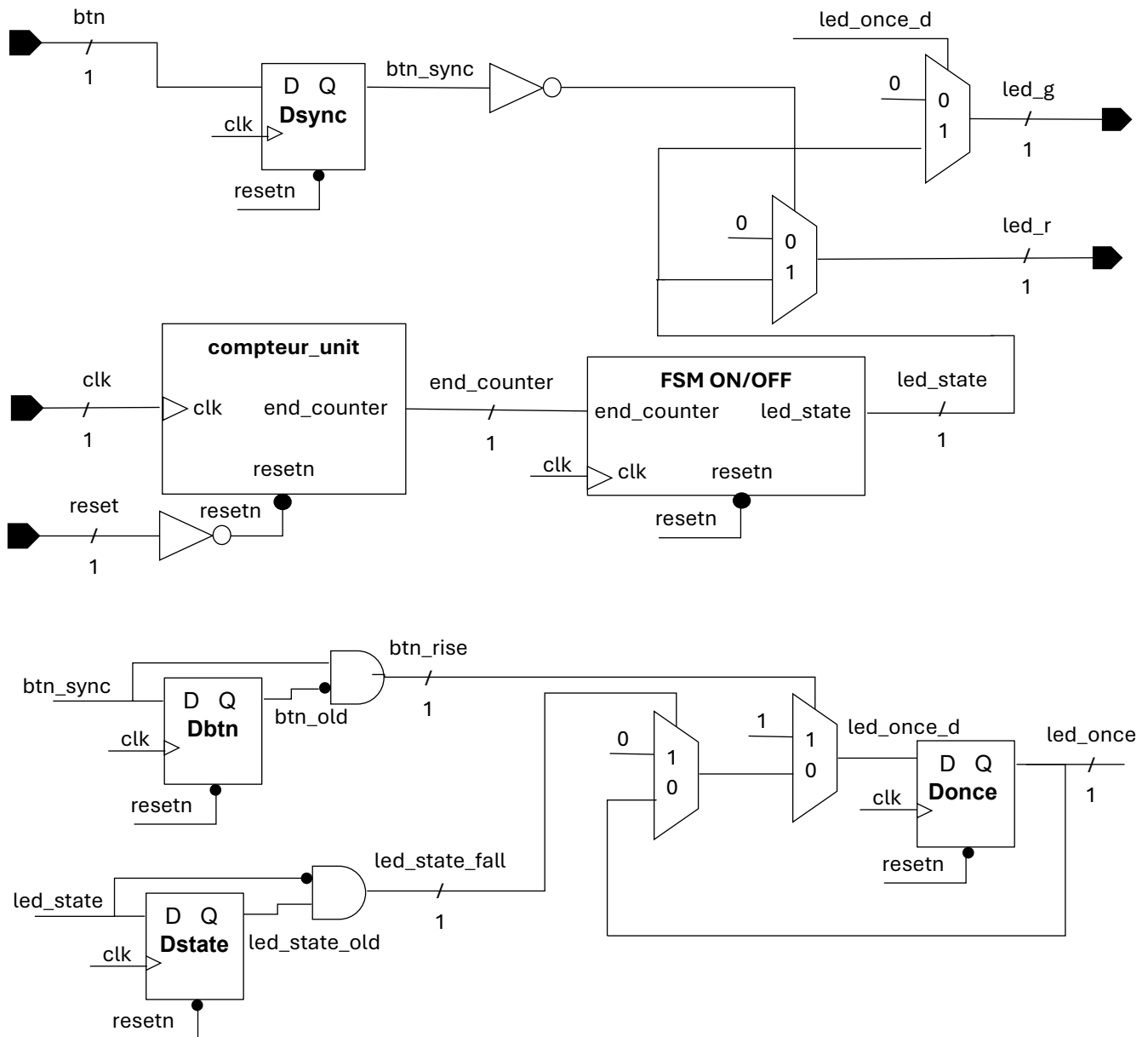
4 suite . Que se passe-t-il si le bouton est pressé pendant plus d'un cycle d'horloge ?

Si le bouton est appuyé plus d'un coup d'horloge , la LED clignote en vert en suivant les cycles led_on/led_off

5. Que faudrait-il faire pour que la LED ne clignote en vert qu'une seule fois même si le bouton est maintenu ?

Pour que la LED ne clignote qu'une fois en vert même si le bouton est maintenu, il faut détecter le front montant sur btn au lieu de détecter un niveau à '1'. Ensuite il faut mémoriser que l'on a déjà fait un cycle d'allumage pour ne pas allumer la LED en vert sur le deuxième cycle même si le bouton est appuyé.

6. Ajoutez cette solution à votre architecture RTL.



- Une bascule `Dbtn` et une porte ET ont été rajouté pour détecter le front montant du bouton `btn`
- une bascule `Dsync` a été rajoutée pour synchroniser le signal `btn` sur l'horloge. Sans cette synchronisation, il y avait un risque de ne pas détecter l'appui sur le bouton (asynchrone) dans le cas où celui-ci se produisait exactement sur un front d'horloge
- Une bascule `Dstate` et une porte ET ont été rajouté pour détecter le front descendant du signal `led_state` qui contrôle l'allumage de la LED
- Le signal `led_once` a été rajouté, il sera à 1 sur le prochain cycle d'allumage de la LED après qu'un front montant ait été détecté sur `btn`.

Le premier multiplexeur connecté à la bascule est drivé par `btn_rise`. Le second multiplexeur est drivé par `led_state_fall`. la condition `btn_rise` est prioritaire sur `led_state_fall` pour éviter d'activer la LED allumée en vert si l'appui du bouton est effectué en même temps qu'un front descendant du signal `led_state`. Sinon on pourrait rater l'allumage en vert sur l'appui du bouton.

- Le signal `led_once_d` est utilisé pour piloter le multiplexeur sur la LED verte. Nous utilisons `led_once_d` au lieu de `led_once` pour avoir une continuité de l'allumage de la LED à la bascule entre rouge et vert. Si on utilisais `led_once` qui arrive un coup d'horloge après `btn_sync`, le signal `led_r` passe à 0 un coup d'horloge avant que `led_g` passe à 1

7. Mettez à jour votre code VHDL et vérifiez votre résultat à la simulation.

- code VHDL mis à jour

Le fichier est tp_led_driver.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tp_led_driver is
    generic (
        cible : integer := 20
    );
    port (
        clk      : in std_logic;
        reset    : in std_logic;
        btn      : in std_logic;
        led_r    : out std_logic;
        led_g    : out std_logic
    );
end tp_led_driver;

architecture behavioral of tp_led_driver is

    type state is (led_on, led_off);
    signal current_state : state;
    signal next_state : state;
    signal resetn : std_logic := '0';
    signal end_counter : std_logic := '0';
    signal led_state : std_logic := '0';
    signal btn_sync : std_logic := '0';
    signal btn_old : std_logic := '0';
    signal btn_rise : std_logic := '0';
    signal led_state_old : std_logic := '0';
    signal led_state_fall : std_logic := '0';
    signal led_once : std_logic := '0';
    signal led_once_d : std_logic := '0';

begin
    cpt: entity work.counter_unit(behavioral)
        generic map (
            cible => cible
        )
        port map (
            clk => clk,
            resetn => resetn,
            end_counter => end_counter
        );
```

Appel du compteur de cycle
counter_unit


```

process(clk, resetn)
begin
    if resetn = '0' then
        current_state <= led_off;
        btn_sync <= '0';
        btn_old <= '0';
        led_state_old <= '0';
        led_once <= '0';
    elsif rising_edge(clk) then
        current_state <= next_state;
        btn_sync <= btn;
        btn_old <= btn_sync;
        led_state_old <= led_state;
        led_once <= led_once_d;
    end if;
end process;
process(current_state, end_counter)
begin
    next_state <= current_state;
    case current_state is
        when led_off =>
            if end_counter = '1' then
                next_state <= led_on;
            end if;
        when led_on =>
            if end_counter = '1' then
                next_state <= led_off;
            end if;
    end case;
end process;

resetn <= not reset;
btn_rise <= '1' when btn_old = '0' and btn_sync = '1' else '0';
led_state_fall <= '1' when led_state_old = '1' and led_state = '0' else '0';
led_once_d <= '1' when btn_rise = '1'
    else '0' when led_state_fall = '1'
    else led_once;
led_state <= '1' when current_state = led_on else '0';
led_r <= led_state when btn_sync = '0' else '0';
led_g <= led_state when led_once_d = '1' else '0';
end behavioral;

```

reset des bascules

logique séquentielle des bascules

Process séquentiels de contrôle de la machine d'état onoff

Process combinatoire
 détection du front montant sur btn
 détection du front descendant sur l'état de la LED
 génération du contrôle d'un seul clignotement sur led_g
 génération des signaux de contrôle des couleurs des LED

- Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn, btn

Liste des signaux de sortie à vérifier : led_r, led_g

Liste des signaux internes à vérifier : end_counter, led_state, btn_rise, led_state_fall; led_once

Listes des états à vérifier: current_state, next_state

- Déroulé de la validation et du test bench

Le déroulé de la validation et le test bench est identique à la validation du circuit précédent (question 2 à 4) excepté pour le test de l'appui sur le bouton . Je vais repréciser et modifier ce test comme indiqué ci-dessous

3. Test de l'appui sur le bouton

3.1. Les entrées seront définies comme suit :

- resetn = 1
- **btn = 1**

3.2. Les sorties seront vérifiées

- led_r = 0 ,
- **led_g = 0/1/0** sur le premier cycle de comptage du compteur_unit suivant l'appui sur le bouton

3.3. Les signaux : états internes à vérifier

- led_state = 0/1 à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*periode(clk) secondes)
- end_counter passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)
- current_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*periode(clk) secondes)
- btn_rise va passer à 1 sur un coup d'horloge suivant l'appui sur le bouton (détection de front montant de btn)
- led_fall va passer à 1 sur la fin de comptage suivant l'appui sur le bouton (détection de front descendant de led_state)
- led_once va passer à 1 sur btn_rise puis passer à 0 sur led_fall

3.4. Sur le test bench

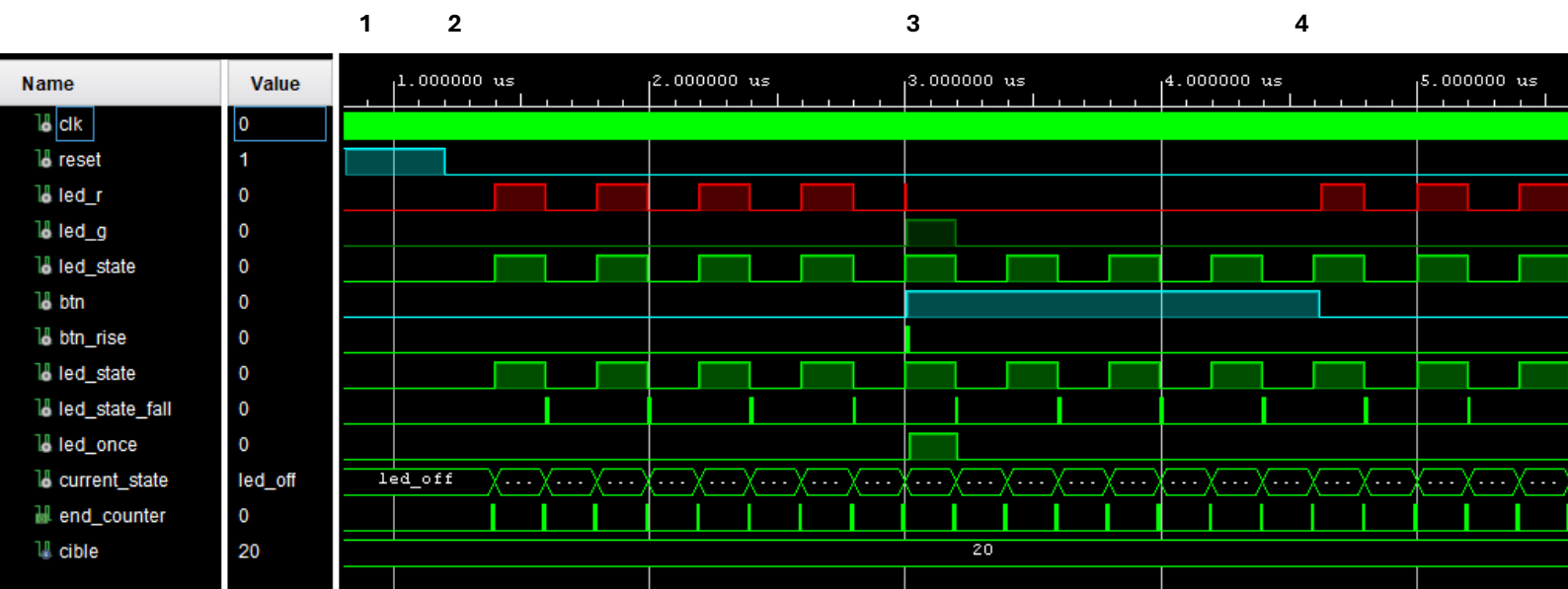
- on attend 1*periode(clk)+1ns
- on teste les 2 sorties de led : led_r=0, led_g = 1
- on attend 2*cible*periode(clk)+1ns
- on teste les 2 sorties de led : led_r=0, led_g = 0
- on attend 4*2*cible*periode +1ns pour lancer la suite

Le test bench complet est décrit dans le fichier tb_tp_led_driver.vhd

- Résultat des test automatiques du test_bench

```
# run 8000ns
Note: PASS : TEST reset : la led est eteinte OK
Time: 1001 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
Note: PASS : TEST demarrage après reset avec btn non appuyé : led en rouge OK
Time: 1403 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
Note: PASS : TEST btn appuyé : led en vert sur le premier cycle d'allumage OK
Time: 3015 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
Note: PASS : TEST btn appuyé : led éteinte sur les cycles d'allumage suivants OK
Time: 3416 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
Note: PASS : TEST btn relaché : led en rouge OK
Time: 4020 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
Note: Nombre de test FAIL : 0 AUCUN TEST FAILED
Time: 6229 ns Iteration: 0 Process: /tb_tp_led_driver/line__49 File: D:/Vivado/TP4_LED_MEM/tb_tp_led_driver.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_tp_led_driver_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 8000ns
} launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:08 . Memory (MB): peak = 3017.188 ; gain = 0.309
```

- Validation fonctionnelle



1- Test du reset: la LED est éteinte (led_r = 0 et led_g = 0)

le compteur_unit est resetté (end_counter = 0)

la FSM est dans l'état led_off

2- Démarrage après reset: le counter_unit démarre

la FSM bascule de led_on à led_off tous à chaque fin de cycle de comptage (end_counter = 1)

La LED est en rouge car le bouton n'est pas appuyé (btn = 0), elle clignote selon le cycle de la FSM

3- Appui sur le bouton (btn = 1)

le counter_unit continue de compter

la FSM continue de basculer de led_on à led_off

La LED est en vert sur le premier cycle de la FSM puis elle reste éteinte tant que le bouton est appuyé

4- On relâche le bouton (btn = 0)

le counter_unit continue de compter

la FSM continue de basculer de led_on à led_off

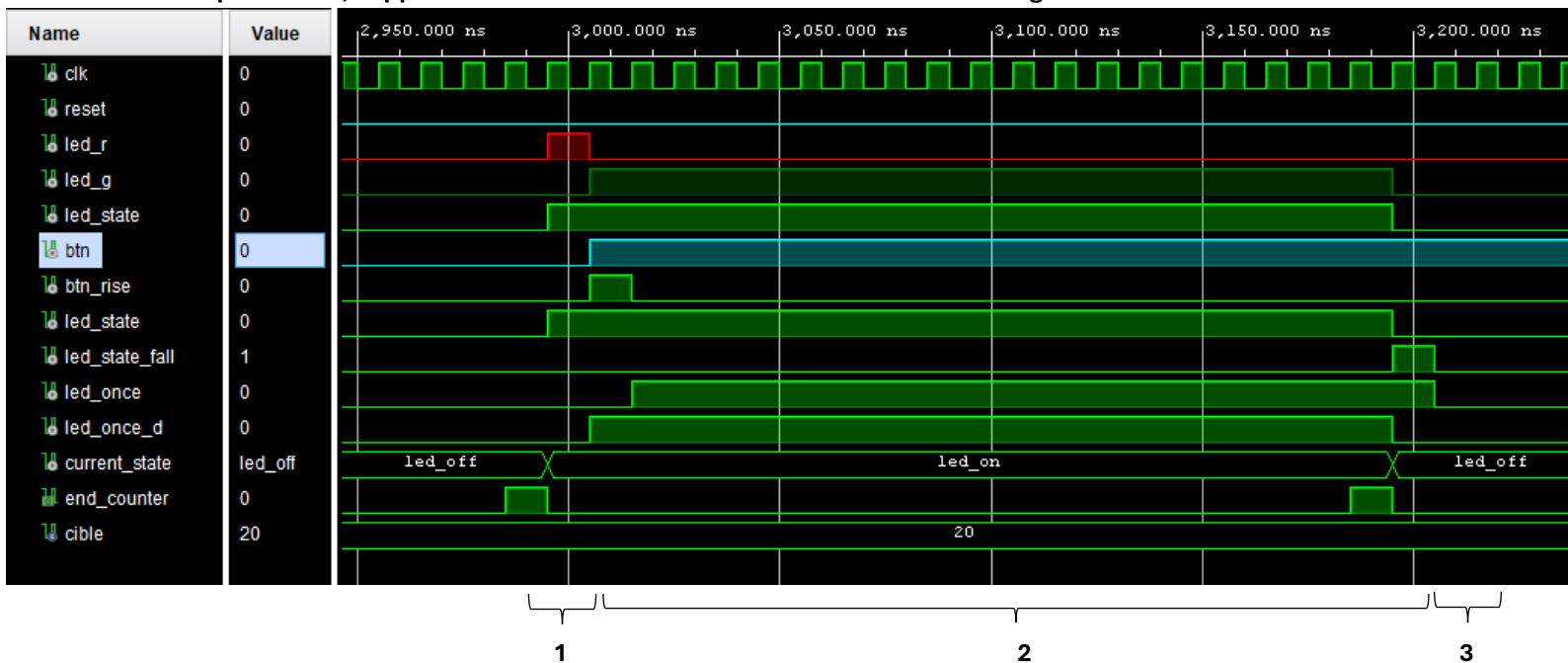
La LED est en rouge car le bouton est relâché (btn = 0), elle clignote selon le cycle de la FSM

Les partie 1 2 et 4 sont inchangée . Nous allons vérifier le nouveau comportement sur la partie 3

- Validation de l'appui sur le bouton btn

La validation a été faite dans plusieurs configurations d'appui sur le bouton

Cas1: cas particulier, l'appui sur le bouton a lieu sur le front montant de l'horloge



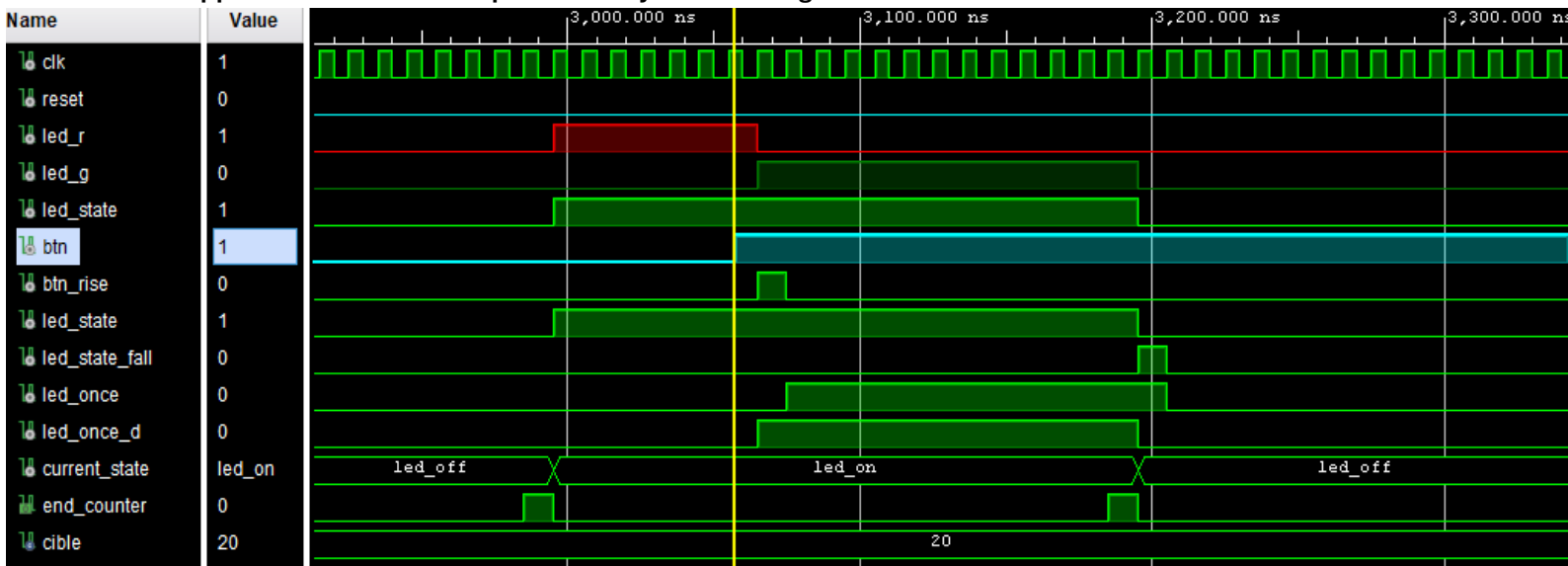
1 Le front montant est détecté : btn_rise passe à 1 pendant un coup d'horloge

2 le cycle de comptage en cours est détecté: led_once passe à 1

led_g suit led_once_d pour ne pas avoir de coupure dans l'éclairage de la LED entre le passage de rouge à vert

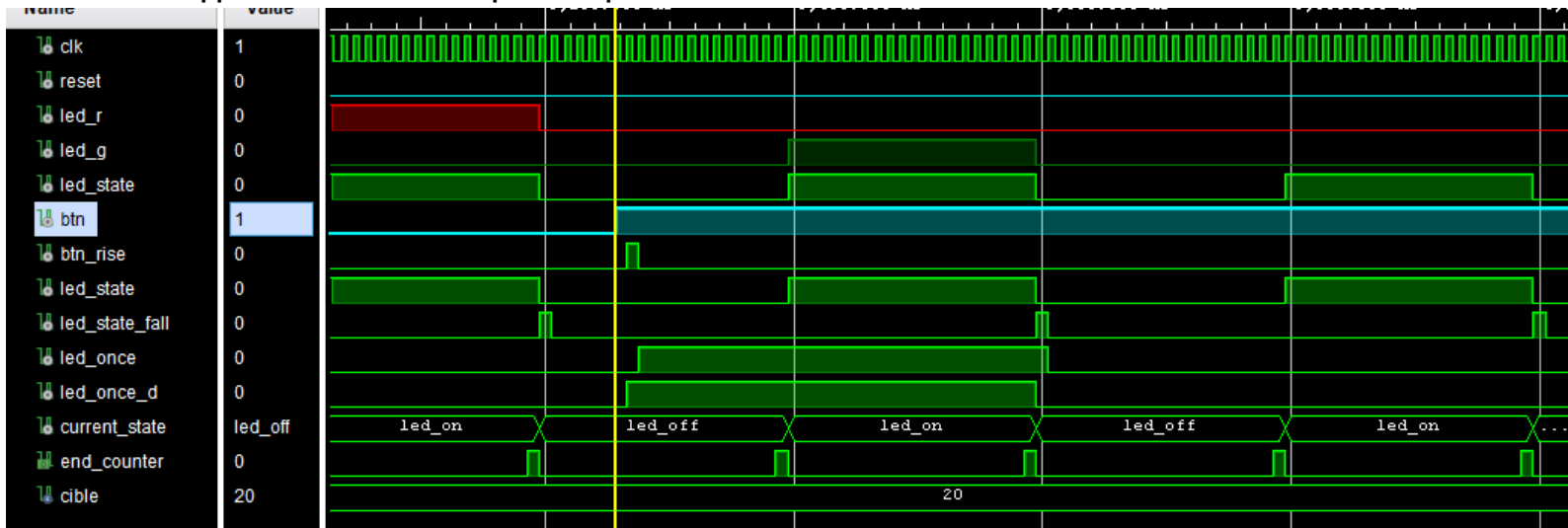
3 à la fin de cycle de comptage led_once repasse à 0 led_g aussi

Cas 2 L'appui sur le bouton a lieu pendant le cycle d'éclairage de la LED



L'éclairage en vert bascule le coup d'horloge suivant l'appui sur le bouton sans coupure d'éclairage

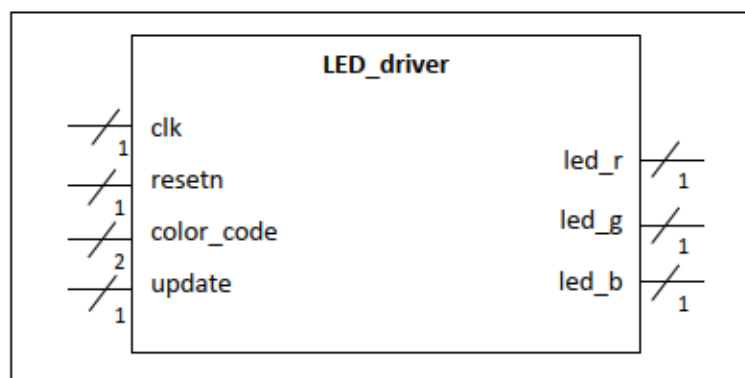
Cas 3 L'appui sur le bouton a lieu pendant que la LED est éteinte



L'éclairage s'allume en vert lors du prochain cycle d'éclairage (current state = led_on)

Elle ne se rallume pas sur les prochains cycles d'éclairage tant que le bouton est appuyé.

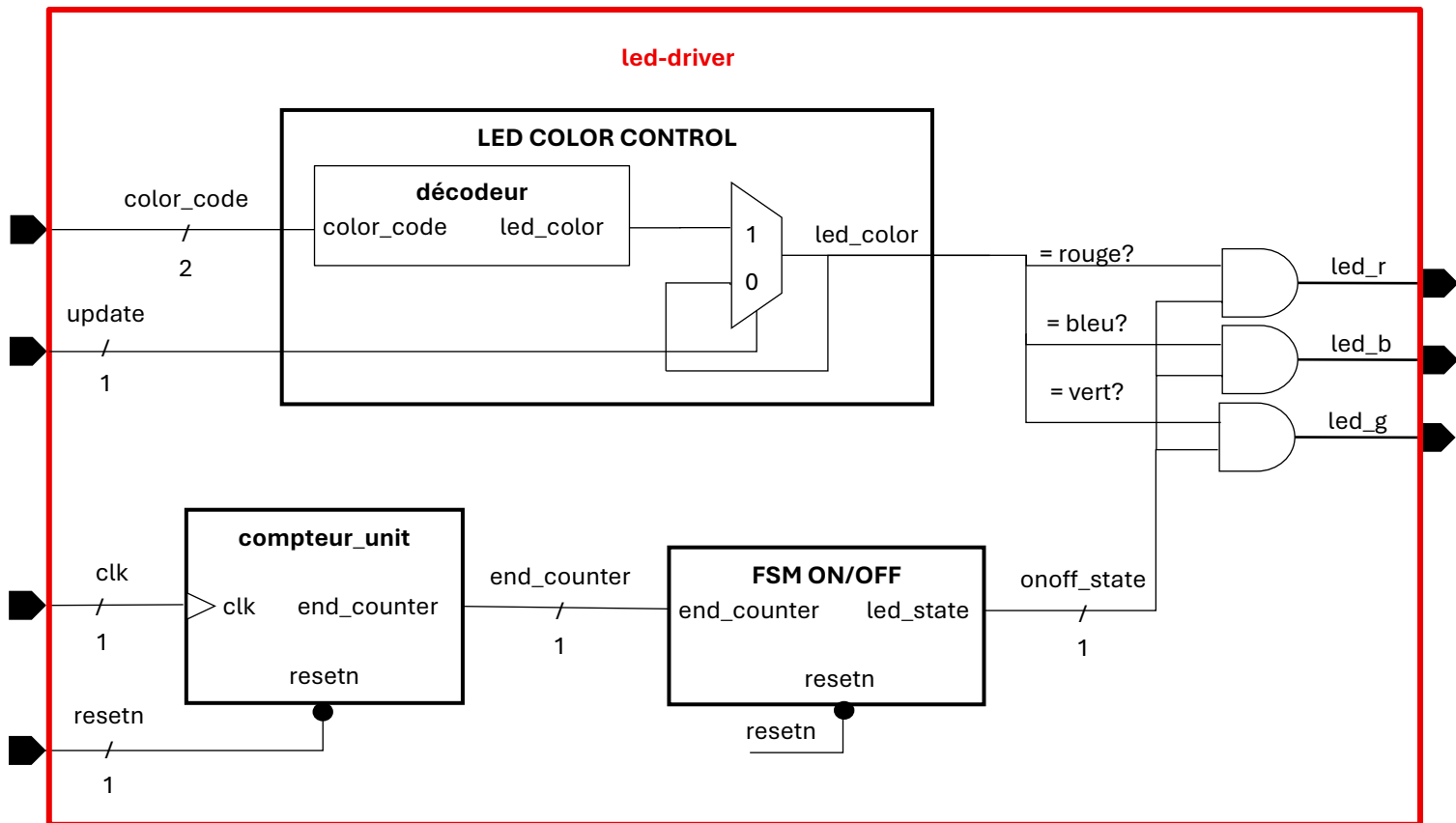
8. Créez un module de pilotage d'une LED RGB en RTL. Ce dernier doit permettre de faire clignoter une LED RGB connectée en sortie d'une couleur définie par un code couleur donné en entrée. Le changement de couleur de la LED RGB n'a lieu que si un signal update est reçu.



A titre d'exemple voici une table mettant en relation un code couleur avec une couleur de la LED RGB :

Code couleur	Couleur de la LED RGB
01	Rouge
10	Verte
11	Bleue
00	Eteinte

- Schéma RTL du module de pilotage



- On reprend l'architecture compteur_unit et FSM ON:OFF pour gérer le clignotement de la LED.
- On rajoute un bloc pour décoder le code couleur et mettre à jour la couleur de la LED uniquement quand le signal `update = 1`
 - le décodeur assigne un état au signal `led_color` (off, rouge, bleu ou vert) en fonction du `color_code`.
 - le signal `update` contrôle un multiplexeur qui met à jour `led_color` si `update = 1` sinon il garde la valeur enregistrée
 - le signal `resetn` met le LED en off; le `resetn` est prioritaire sur l'`update`.

- code VHDL de l'architecture

Le fichier est led_driver.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

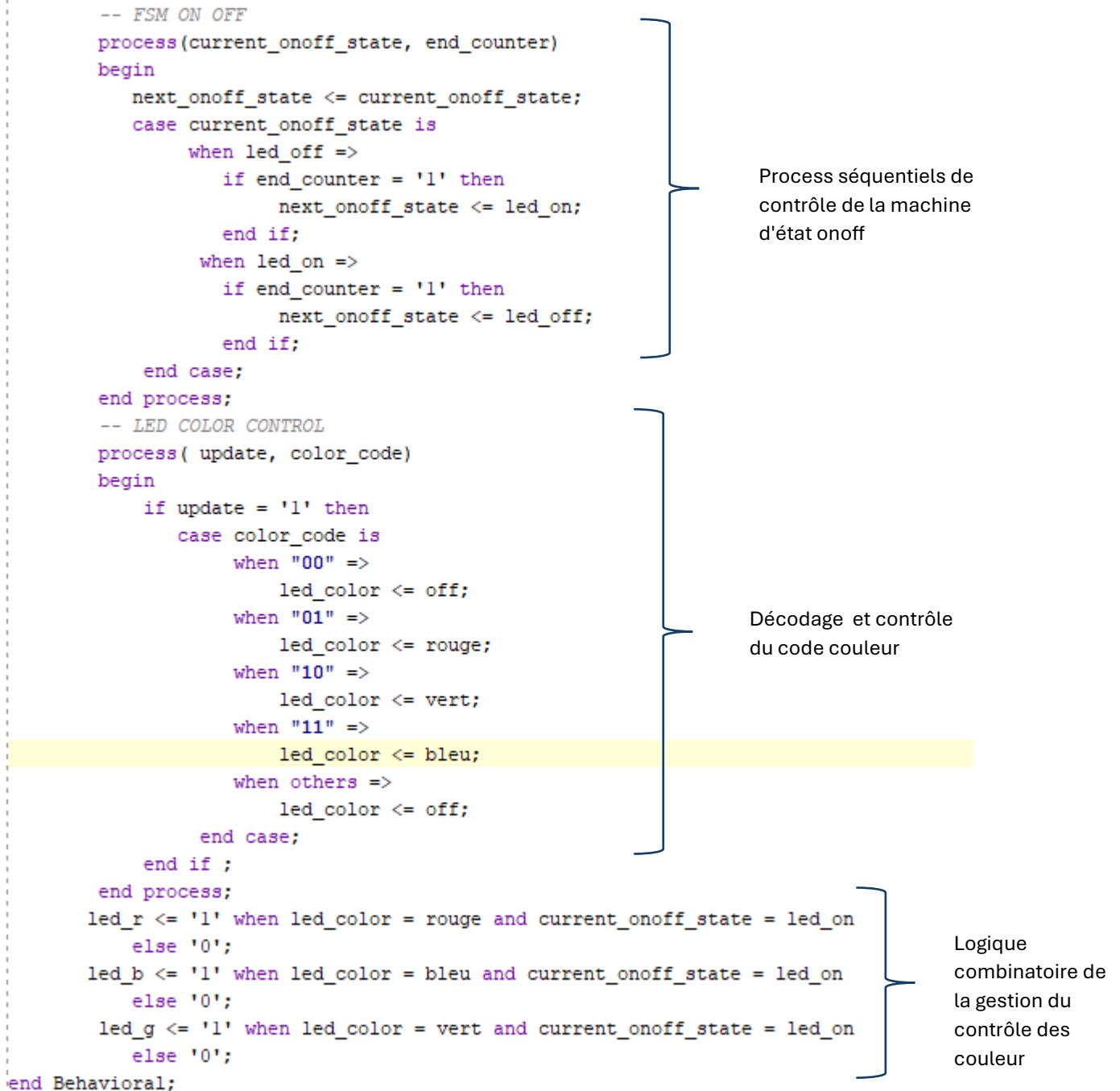
entity led_driver is
    generic (
        cible : integer := 20
    );
    Port ( clk : in STD_LOGIC;
          resetn : in STD_LOGIC;
          color_code : in STD_LOGIC_VECTOR (1 downto 0);
          update : in STD_LOGIC;
          led_r : out STD_LOGIC;
          led_g : out STD_LOGIC;
          led_b : out STD_LOGIC);
end led_driver;

architecture Behavioral of led_driver is
    type onoff_state is (led_on, led_off);
    signal current_onoff_state : onoff_state;
    signal next_onoff_state : onoff_state;
    type color_state is (off, rouge, bleu, vert);
    signal led_color : color_state;
    signal end_counter: std_logic:='0';

    begin
        cpt: entity work.counter_unit (behavioral)
            generic map (
                cible => cible
            )
            port map (
                clk => clk,
                resetn => resetn,
                end_counter => end_counter
            );
        process (clk, resetn)
        begin
            if resetn = '0' then
                current_onoff_state <= led_off;
            elsif rising_edge(clk) then
                current_onoff_state <= next_onoff_state;
            end if ;
        end process;
    end architecture;

```

Appel du compteur de cycle
counter_unit



- Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn, update, color_code

Liste des signaux de sortie à vérifier : led_r, led_b, led_g

Liste des signaux internes à vérifier : end_counter, led_color

Listes des états à vérifier: current_onoff_state, next_onoff_state

L'horloge clk est définie à 100Mhz

Note : les simulations ont été faite avec une cible de 20 soit un temps de comptage du compteur unitaire de 200ns

- Déroulé de la validation et du test bench

1. Test du reset au démarrage

- 1.1. Les entrées seront définies comme suit : :
 - **resetn = 0**
 - **update = 0**
 - **color_code = 00**
- 1.2. Les sorties seront vérifiées
 - **led_r = 0 , led_b = 0 , led_g = 0**
- 1.3. Les signaux : états internes à vérifier
 - **led_state = 0** resetté
 - **end_counter = 0** : counter_unit resetté
 - **led_color = off** : led color control initialisé à off
- 1.4. Sur le test bench
 - on teste les 3 sorties de led : led_r=0,led_b=0, led_g=0 après $(1*(2*cible)+cible)*periode(clk)+1ns$
 - on attend $cible*period + 1ns$ pour lancer la suite

2. Test du démarrage après reset

- 2.1. Les entrées seront définies comme suit : :
 - **resetn = 1**
 - **update = 0**
 - **color_code = 00**
- 2.2. Les sorties seront vérifiées
 - **led_r = 0 , led_b = 0 , led_g = 0**
- 2.3. Les signaux : états internes à vérifier
 - **led_color** est off , la LED est éteinte
 - **end_counter** passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)
 - **current_onoff_state = led_off/led_on** à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)
- 2.4. Sur le test bench
 - on teste les sorties de led : led_r=0,led_b=0, led_g=0 après $cible*periode(clk)+1ns$
 - on attend $2*2*cible*period + 1ns$ pour lancer la suite

3. Test de chaque code couleur avec son update

les étapes 1 à 3 seront répétées pour chaque code couleur

Je ne décris ici qu'une seule séquence pour le code '01': led rouge

Etape 1 : on applique le code couleur

- 3.1. Les entrées seront définies comme suit : :
 - **resetn = 1**
 - **color_code = '01'**
 - **update = '0'**
- 3.2. Les sorties seront vérifiées
 - **led_r, led_b, led_g** ne changent pas de fonctionnement
- 3.3. Les signaux : états internes à vérifier
 - **led_color** ne change pas d'état, il reste dans l'état où il était avant le changement de code couleur
 - **end_counter** passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)
 - **current_onoff_state = led_off/led_on** à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)
- 3.4. Sur le test bench
 - on teste les sorties de led : led_r, led_b, led_g après 1ns: elle ne doivent pas avoir changé de comportement
 - on attend $2*2*cible*period$ pour lancer la suite

4. Test de chaque code couleur avec son update

Etape 2 : on applique l'update

- 4.1. Les entrées seront définies comme suit : :
 - **resetn = 1**
 - **color_code = '01'**
 - **update = '1'**

4.2. Les sorties seront vérifiées

- **led_r= 0/1** à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)
- **led_b = '0', led_g = '0'**

4.3. Les signaux : états internes à vérifier

- **led_color = rouge**
- **end_counter** passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)
- **current_onoff_state = led_off/led_on** à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)

4.4. Sur le test bench

- on teste les sorties de led : led_r = 1 , led_b= 0 , led_g= 0
- on attend $2*2*cible*period$ pour lancer la suite

5. Test de chaque code couleur avec son update

Etape 3 : on relâche l'update

5.1. Les entrées seront définies comme suit :

- **resetn = 1**
- **color_code = '01'**
- **update = '0'**

5.2. Les sorties seront vérifiées

- **led_r= 0/1** à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)
- **led_b = '0', led_g = '0'**

5.3. Les signaux : états internes à vérifier

- **led_color = rouge**
- **end_counter** passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)
- **current_onoff_state = led_off/led_on** à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)

5.4. Sur le test bench

- on teste les sorties de led : led_r = 1 , led_b= 0 , led_g= 0
- on attend $2*2*cible*period$ pour lancer la suite

6. Test du reset sur un code couleur rouge

Nous avons auparavant rallumé la Led en rouge : color_code = '01' puis update = 1

6.1. Les entrées seront définies comme suit :

- **resetn = 0**
- **update = 1**
- **color_code = 01**

6.2. Les sorties seront vérifiées

- **led_r = 0 , led_b = 0 , led_g= 0**

6.3. Les signaux : états internes à vérifier

- **led_state = led_off** resetté
- **end_counter = 0 : counter_unit** resetté
- **led_color = rouge** : led color control n'est mis à jour que par l'update, pas de reset

6.4. Sur le test bench

- on teste les 3 sorties de led : led_r =0, led_b= 0, led_g = 0 après 1ns

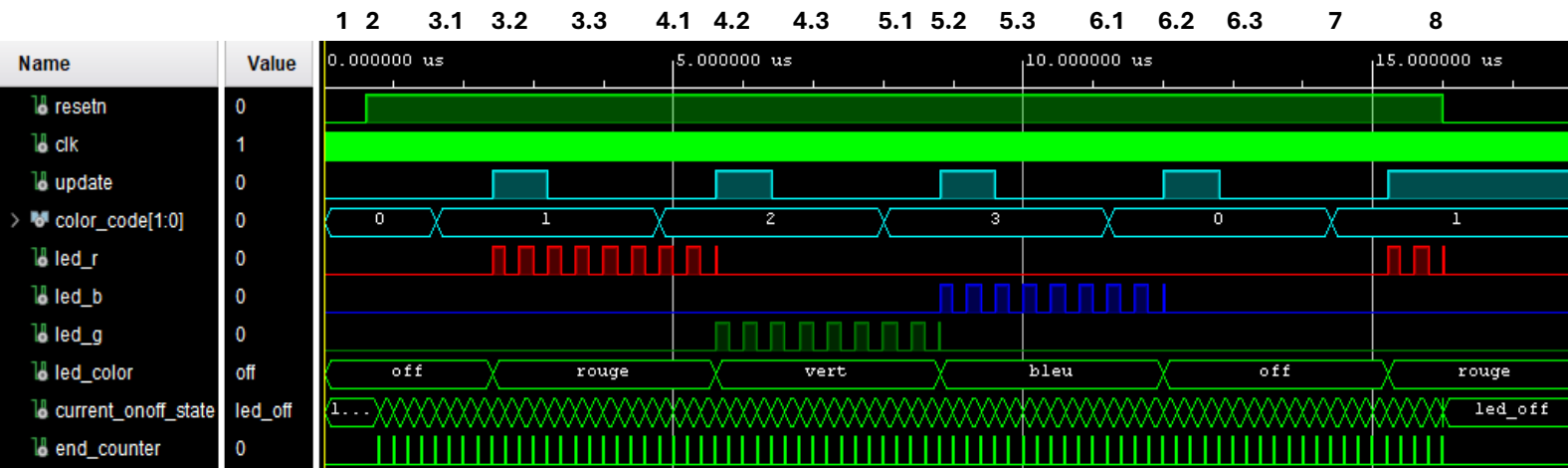
• Test bench

Le fichier de test bench est tb_led_driver.vhd

- Résultat des test automatiques du test_bench

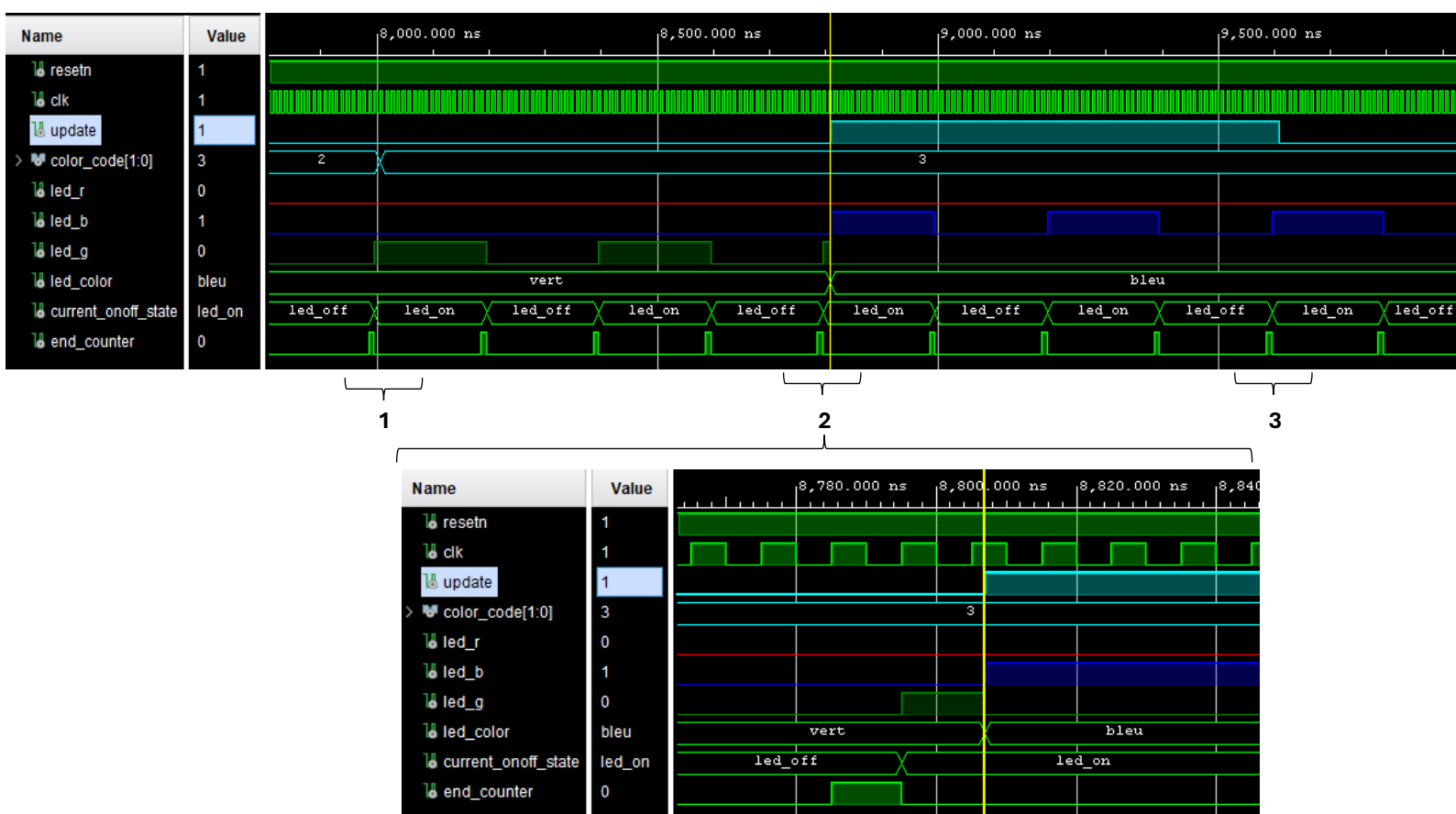
```
# run 18us
Note: PASS : TEST reset : la led est eteinte OK
Time: 601 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST demarrage : la led est eteinte OK
Time: 802 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST code led rouge sans update : la led est eteinte OK
Time: 1603 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led rouge : la led est rouge OK
Time: 2404 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST update => 0 pour code led rouge : la led reste rouge OK
Time: 3204 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST code led verte sans update : la led reste rouge OK
Time: 4805 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led verte : la led est verte OK
Time: 5606 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST update => 0 pour code led verte : la led reste verte OK
Time: 6406 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST code led bleue sans update : la led reste verte OK
Time: 8007 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led bleue : la led est bleue OK
Time: 8808 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST update => 0 pour code led bleue : la led reste bleue OK
Time: 9608 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST code led off sans update : la led reste bleue OK
Time: 11209 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led off : la led est eteinte OK
Time: 12010 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST update => 0 pour code led off : la led reste eteinte OK
Time: 12810 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST code led rouge sans update : la led est eteinte OK
Time: 14411 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led rouge : la led est rouge OK
Time: 15212 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: PASS : TEST resetn => 0 : la led est eteinte OK
Time: 16013 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
Note: Nombre de test FAIL : 0 AUCUN TEST FAILED
Time: 16813 ns Iteration: 0 Process: /tb_led_driver/line__54 File: D:/Vivado/TP4_LED_MEM/tb_led_driver.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_led_driver_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 18us
```

- Validation fonctionnelle



- 1- Test du reset: la LED est éteinte (led_r = 0 , led_b= 0 , led_g = 0)
 Le compteur_unit est resetté (end_counter = 0)
 La FSM (current_onoff_state)est dans l'état led_off
 La couleur de led est off (led_color = off)
- 2- Démarrage après reset: le counter_unit démarre
 la FSM(current_onoff_state) bascule de led_on à led_off à chaque fin de cycle de comptage (end_counter = 1)
 La LED est off (led_color = off)
- 3.1- On applique le code couleur rouge
 Rien ne change : la led est off (led_color = off)
- 3.2- On applique le update à 1
 La LED clignote en rouge (led_r bascule de 0 à 1 , led_color = rouge)
- 3.3- On applique le update à 0
 La LED continue à clignoter en rouge (led_r bascule de 0 à 1 , led_color = rouge)
- 4.1- On applique le code couleur vert
 Rien ne change : La LED continue à clignoter en rouge
- 4.2- On applique le update à 1
 La LED clignote en vert (led_g bascule de 0 à 1 , led_color = vert)
- 4.3- On applique le update à 0
 La LED continue à clignoter en vert (led_g bascule de 0 à 1 , led_color = vert)
- 5.1- On applique le code couleur bleu
 Rien ne change : La LED continue à clignoter en vert
- 5.2- On applique le update à 1
 La LED clignote en bleu (led_b bascule de 0 à 1 , led_color = bleu)
- 5.3- On applique le update à 0
 La LED continue à clignoter en bleu (led_b bascule de 0 à 1 , led_color = bleu)
- 6.1- On applique le code couleur off
 Rien ne change : La LED continue à clignoter en bleu
- 6.2- On applique le update à 1
 La LED est éteinte (led_color = off)
- 6.3 On applique le update à 0
 La LED reste éteinte (led_color = off)
- 7 – On rallume la LED en rouge avant d'appliquer un reset
- 8 – On applique un reset alors que la LED clignote en rouge
 La LED s'éteint
 Le compteur_unit est resetté (end_counter = 0)
 La FSM (current_onoff_state)est dans l'état led_off
 La couleur de led reste à rouge (led_color = rouge)

- Validation d'un changement de couleur (vert vers bleu)



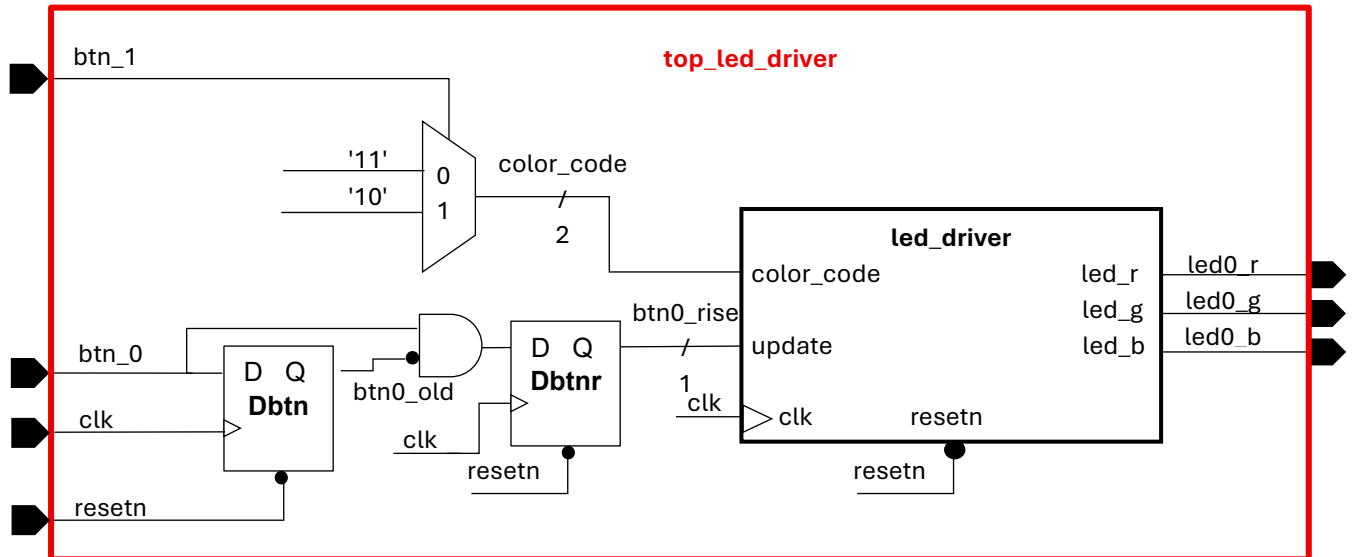
- 1-Au changement de valeur de color_code : le comportement de la LED reste inchangé
- 2- Dès que update passe à 1 , la LED change de couleur de façon asynchrone en suivant la valeur de led_color
ici led_g = 0 et led_b = 1
- 3- Quand update passe à 0: la LED continue de clignoter selon de color_code appliqué
(ici led_b = 0/1, led_g = 0 , led_r = 0)

- Conclusion

Le module led_driver fonctionne comme prévu : il permet de faire clignoter une LED d'une couleur définie par un code couleur en entrée . Le changement de couleur n'a lieu que si un signal update est reçu

9. Ajouter la logique nécessaire pour piloter les entrées/sorties de votre module.

- Le signal update doit recevoir 1 uniquement lorsque le bouton _0 vient d'être appuyé, maintenir le bouton enfoncé ne doit pas maintenir le signal update à 1
- Le signal color_code doit recevoir soit le code couleur « vert » si le bouton_1 est pressé recevoir le code couleur « bleu » sinon
- Les LED R, G et B sont connectées avec les couleurs respectives de la LED_0



- on réutilise le led_driver et on contrôle ses entrées selon les consignes
- le signal update est connecté à la détection du front montant sur le bouton btn_0 ; il sera actif pendant une période d'horloge.
- le color_code est choisi grâce à un multiplexeur dirigé par le bouton btn_1.

10. Ecrivez le code VHDL correspondant à votre architecture.

le fichier est top_led_driver.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

entity top_led_driver is
    generic (
        cible : integer := 200000000
    );
    port (
        clk      : in std_logic;
        resetn   : in std_logic;
        btn0     : in std_logic;
        btn1     : in std_logic;
        led0_r   : out std_logic;
        led0_b   : out std_logic;
        led0_g   : out std_logic
    );
end top_led_driver;

architecture behavioral of top_led_driver is
    signal color_code : STD_LOGIC_VECTOR (1 downto 0) := "00";
    signal btn0_old : std_logic := '0';
    signal btn0_rise : std_logic := '0';

begin
    led_driver: entity work.led_driver(behavioral)
        generic map (
            cible => cible
        )
        port map (
            clk => clk,
            resetn => resetn,
            color_code => color_code,
            update => btn0_rise,
            led_r => led0_r,
            led_g => led0_g,
            led_b => led0_b
        );
    process(clk, resetn)
    begin
        if resetn = '0' then
            btn0_old <= '0';
            btn0_rise <= '0';
        elsif rising_edge(clk) then
            btn0_old <= btn0;
            if btn0_old = '0' and btn0 = '1' then
                btn0_rise <= '1';
            else
                btn0_rise <= '0';
            end if;
        end if;
    end process;
    color_code <= "10" when btn1 = '1' else "11";
end behavioral;
```

Appel de led_driver

Détection de l'appui sur btn0

codage du code couleur en fonction de btn1

11. Ecrivez un testbench pour tester votre design.

- Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn, btn0, btn1

Liste des signaux de sortie à vérifier : led_r, led_b, led_g

Liste des signaux internes à vérifier : color_code, btn0_rise, led_driver: led_color

Listes des états à vérifier: led_driver: current_onoff_state

L'horloge clk est définie à 100Mhz

Note : les simulations ont été faite avec une cible de 20 soit un temps de comptage du compteur unitaire de 200ns

- Déroulé de la validation et du test bench

1. Test du reset au démarrage

1.1. Les entrées seront définies comme suit :

- **resetn = 0**
- btn0 = 0
- btn1 = 0

1.2. Les sorties seront vérifiées

- led_r = 0 , led_b = 0 , led_g = 0

1.3. Les signaux : états internes à vérifier

- color_code = '11' (valeur pour btn1 =0)
- btn0_rise = 0
- led_driver: led_color = off lors du reset
- led_driver: current_onoff_state = led_off resetté

1.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0, led_b= 0, led_g = 0 après $(1 \cdot (2 \cdot \text{cible}) + \text{cible}) \cdot \text{periode}(\text{clk}) + 1\text{ns}$
- on attend $\text{cible} \cdot \text{period} + 1\text{ns}$ pour lancer la suite

2. Test du démarrage après reset

2.1. Les entrées seront définies comme suit :

- **resetn = 1**
- btn0 = 0
- btn1 = 0

2.2. Les sorties seront vérifiées

- led_r = 0 , led_b = 0 , led_g = 0

2.3. Les signaux : états internes à vérifier

- color_code = '11'
- btn0_rise = 0
- led_driver: led_color = off lors du reset
- led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant $\text{cible} \cdot \text{periode}(\text{clk})$ secondes)

2.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0, led_b= 0, led_g = 0 après $1 \cdot \text{cible} \cdot \text{periode}(\text{clk}) + 1\text{ns}$
- on attend $2 \cdot 2 \cdot \text{cible} \cdot \text{period}$ pour lancer la suite

3. Appui sur btn0

3.1. Les entrées seront définies comme suit :

- resetn = 1
- **btn0 = 1**
- btn1 = 0

3.2. Les sorties seront vérifiées

- led_r = 0 , led_g = 0
- **led_b = 0/1** à chaque fin de comptage du compteur_unit (la LED est allumée pendant $\text{cible} \cdot \text{periode}(\text{clk})$ secondes)

3.3. Les signaux : états internes à vérifier

- color_code = '11'
- **btn0_rise = 1 pendant un coup d'horloge**

- **led_driver: led_color = bleu**
- led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

3.4. Sur le test bench

- on teste les 3 sorties de led : led_r =0, led_b= 1, led_g = 0 après 1ns
- on attend 2*2*cible*period pour lancer la suite

4. Appui sur btn1

4.1. Les entrées seront définies comme suit :

- resetn = 1
- btn0 = 1
- **btn1 = 1**

4.2. Les sorties seront vérifiées(rien ne change)

- led_r = 0 , led_g= 0
- led_b = 0/1 à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

4.3. Les signaux : états internes à vérifier

- color_code = '01'
- btn0_rise = 0
- led_driver: led_color = bleu
- led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

4.4. Sur le test bench

- on teste les 3 sorties de led : led_r =0, led_b= 1, led_g = 0 après period(clk)+1ns
- on attend 2*2*cible*period pour lancer la suite

5. On relâche btn1

5.1. Les entrées seront définies comme suit :

- resetn = 1
- btn0 = 1
- **btn1 = 0**

5.2. Les sorties seront vérifiées(rien ne change)

- led_r = 0 , led_g= 0
- led_b = 0/1 à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

5.3. Les signaux : états internes à vérifier

- color_code = '01'
- btn0_rise = 0
- led_driver: led_color = bleu
- led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

5.4. Sur le test bench

- on teste les 3 sorties de led : led_r =0, led_b= 1, led_g = 0 après period(clk)+1ns
- on attend 2*2*cible*period pour lancer la suite

6. On relâche btn0

6.1. Les entrées seront définies comme suit :

- resetn = 1
- **btn0 = 0**
- btn1 = 0

6.2. Les sorties seront vérifiées

- led_r = 0 , led_g= 0
- **led_b = 0/1** à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

6.3. Les signaux : états internes à vérifier

- color_code = '11'
- **btn0_rise = 0**
- **led_driver: led_color = bleu inchangé**
- led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

6.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0,led_b= 1, led_g = 0 après 1ns
- on attend 2*2*cible*period pour lancer la suite

7. Appui sur btn1

7.1. Les entrées seront définies comme suit :

- resetn = 1
- btn0 = 0
- **btn1 = 1**

7.2. Les sorties seront vérifiées

- led_r = 0 , led_g= 0
- **led_b = 0/1** à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

7.3. Les signaux : états internes à vérifier

- **color_code = '01'**
- btn0_rise = 0
- **led_driver: led_color = bleu**
- led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

7.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0,led_b= 1, led_g = 0 après 1ns
- on attend 2*2*cible*period pour lancer la suite

8. Appui sur btn0

8.1. Les entrées seront définies comme suit :

- resetn = 1
- **btn0 = 1**
- btn1 = 1

8.2. Les sorties seront vérifiées

- led_r = 0 , **led_b= 0**
- **led_g = 0/1** à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

8.3. Les signaux : états internes à vérifier

- color_code = '01'
- **btn0_rise = 1 pendant un coup d'horloge**
- **led_driver: led_color = vert**
- led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

8.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0,led_b= 0, led_g = 1 après period(clk)+1ns
- on attend 2*2*cible*period pour lancer la suite

9. On relâche btn0

9.1. Les entrées seront définies comme suit :

- resetn = 1
- **btn0 = 0**
- btn1 = 1

9.2. Les sorties seront vérifiées

- led_r = 0 , led_b= 0
- **led_g = 0/1** à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

9.3. Les signaux : états internes à vérifier

- color_code = '01'
- **btn0_rise = 0**
- **led_driver: led_color = vert inchangé**
- led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)

9.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0,led_b= 0, led_g = 1 après 1ns
- on attend 2*2*cible*period pour lancer la suite

10. On relâche btn1

- 10.1. Les entrées seront définies comme suit : :
 - `resetn = 1`
 - `btn0 = 0`
 - **`btn1 = 1`**
- 10.2. Les sorties seront vérifiées
 - `led_r = 0` , `led_b = 0`
 - **`led_g = 0/1`** à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
- 10.3. Les signaux : états internes à vérifier
 - `color_code = '01'`
 - `btn0_rise = 0`
 - **`led_driver: led_color = vert inchangé`**
 - `led_driver: current_onoff_state = led_off/led_on` à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
- 10.4. Sur le test bench
 - on teste les 3 sorties de led : `led_r = 0`, `led_b = 0`, `led_g = 1` après 1ns
 - on attend `2*2*cible*period` pour lancer la suite

11. Appui sur btn0

- 11.1. Les entrées seront définies comme suit : :
 - `resetn = 1`
 - **`btn0 = 1`**
 - `btn1 = 0`
- 11.2. Les sorties seront vérifiées
 - `led_r = 0` , `led_g = 0`
 - **`led_b = 0/1`** à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
- 11.3. Les signaux : états internes à vérifier
 - `color_code = '11'`
 - **`btn0_rise = 1 pendant un coup d'horloge`**
 - **`led_driver: led_color = bleu`**
 - `led_driver: current_onoff_state = led_off/led_on` à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
- 11.4. Sur le test bench
 - on teste les 3 sorties de led : `led_r = 0`, `led_b = 1`, `led_g = 0` après `period(clk)+1ns`
 - on attend `2*2*cible*period` pour lancer la suite

12. Test du reset

- 12.1. Les entrées seront définies comme suit : :
 - **`resetn = 0`**
 - `btn0 = 1`
 - `btn1 = 0`
- 12.2. Les sorties seront vérifiées
 - `led_r = 0` , `led_b = 0` , `led_g = 0`
- 12.3. Les signaux : états internes à vérifier
 - `color_code = '11'` (valeur pour `btn1 = 0`)
 - `btn0_rise = 0`
 - `led_driver: led_color = bleu` reste en bleu tant qu'il n'y a pas d'update
 - `led_driver: current_onoff_state = led_off` resetté
- 12.4. Sur le test bench
 - on teste les 3 sorties de led : `led_r = 0`, `led_b = 0`, `led_g = 0` après 1ns

- Test bench

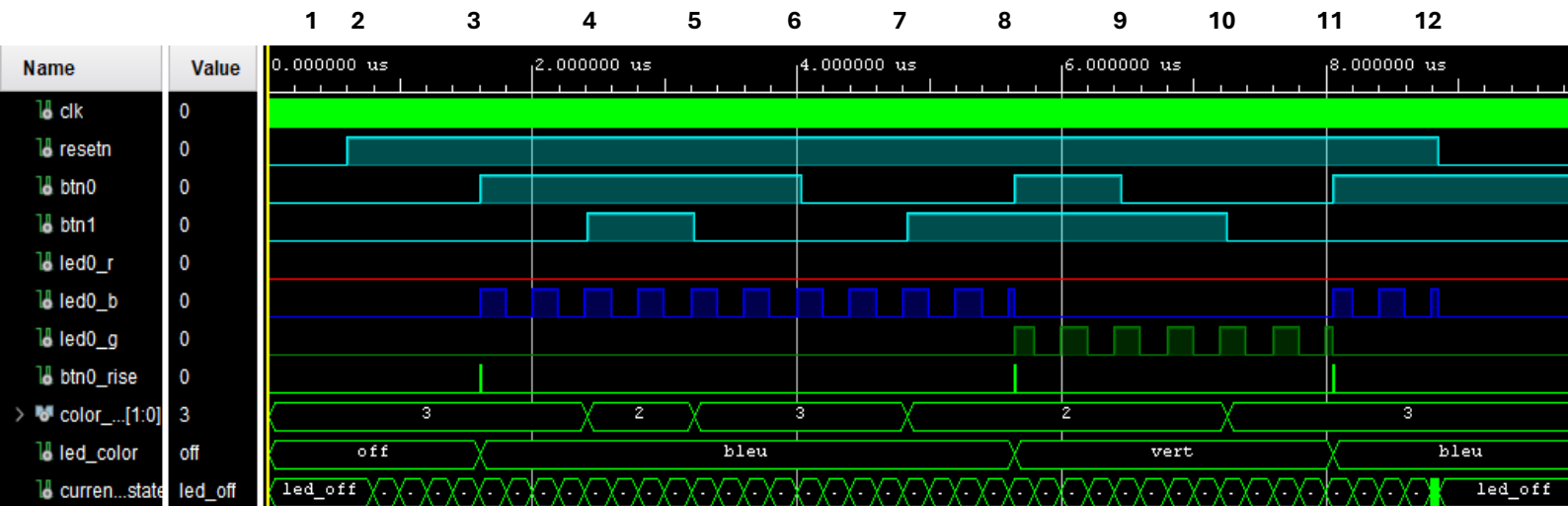
Le fichier de test bench est `tb_top_led_driver.vhd`

12. Vérifier votre résultat à la simulation.

- Résultat des test automatiques du test_bench

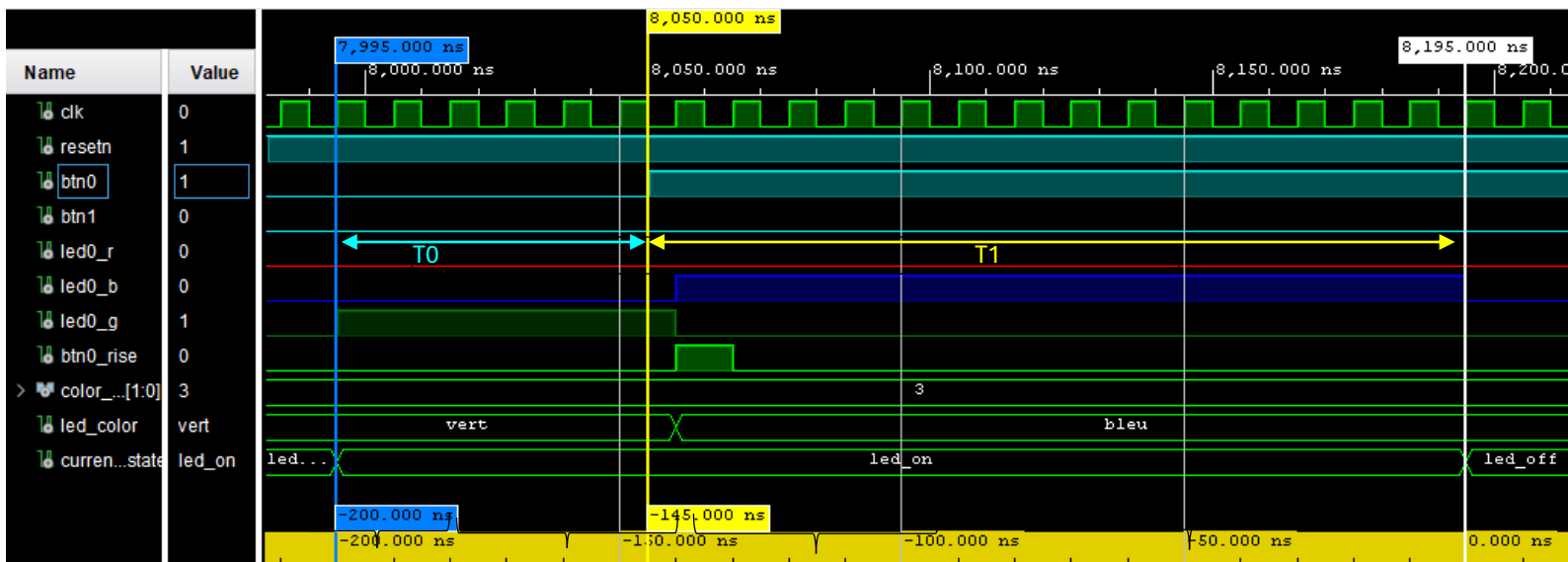
```
# run 10000ns
Note: PASS : TEST reset : la led est eteinte OK
Time: 601 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST demarrage : la led est eteinte OK
Time: 802 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST appui sur btn0 avec btn1 relaché : la led est bleue OK
Time: 1613 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST appui sur btn1 avec btn0 appuyé : la led est bleue OK
Time: 2424 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST appui sur btn1 avec btn0 appuyé : la led est bleue OK
Time: 3235 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST relache btn0 avec btn1 relaché : la led reste bleue OK
Time: 4036 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST appui sur btn1 avec btn0 relaché : la led reste bleue OK
Time: 4837 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST appui sur btn0 avec btn1 appuyé : la led est verte OK
Time: 5648 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST relache btn0 avec btn1 appuyé : la led reste verte OK
Time: 6449 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST relache btn1 avec btn0 relaché : la led reste verte OK
Time: 7250 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST appui sur btn0 avec avec btn1 relaché : la led est bleue OK
Time: 8051 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: PASS : TEST resetn => 0 : la led est eteinte OK
Time: 8852 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
Note: Nombre de test FAIL : 0 AUCUN TEST FAILED
Time: 8852 ns Iteration: 0 Process: /tb_top_led_driver/line__58 File: D:/Vivado/TP4_LED_MEM/tb_top_led_driver.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_top_led_driver_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 10000ns
```

- Validation fonctionnelle



- 1-Test du reset: la LED est éteinte (led_r = 0 , led_b= 0 , led_g = 0)
 La FSM (current_onoff_state)est dans l'état led_off
 La couleur de led est off (led_color = off)
- 2- Démarrage après reset: le counter_unit démarre
 la FSM(current_onoff_state) bascule de led_on à led_off à chaque fin de cycle de comptage (end_counter = 1)
 La LED est off (led_color = off)
- 3-Appui sur btn0
 btn0_rise détecte un front montant et passe à 1.
 Le signal led_color est mis à jour en fonction de la valeur de btn1 => led_color = bleu (color_code = 11)
 le LED clignote en bleu
- 4-Appui sur btn1
 La LED continue de clignoter en bleu, aucun changement
- 5-On relache btn1
 La LED continue de clignoter en bleu, aucun changement
- 6- btn0 est relâché
 La LED continue de clignoter en bleu, aucun changement
- 7- Appui sur btn1
 color_code passe à 01
 La LED continue de clignoter en bleu
- 8-Appui sur btn0
 btn0_rise détecte un front montant et passe à 1.
 Le signal led_color est mis à jour en fonction de la valeur de btn1 => led_color = vert (color_code = 01)
 La LED clignote en vert
- 9- btn0 est relâché
 La LED continue de clignoter en vert, aucun changement
- 10- btn1 est relâché
 La LED continue de clignoter en vert, aucun changement
- 11-Appui sur btn0
 btn0_rise détecte un front montant et passe à 1.
 Le signal led_color est mis à jour en fonction de la valeur de btn1 => led_color = bleu (color_code = 11)
 le LED clignote en bleu
- 12-Reset
 la LED s'éteint
 led_color = off est resetté
 current_onoff_state = led_off est resetté

- Validation de l'appui sur btn0



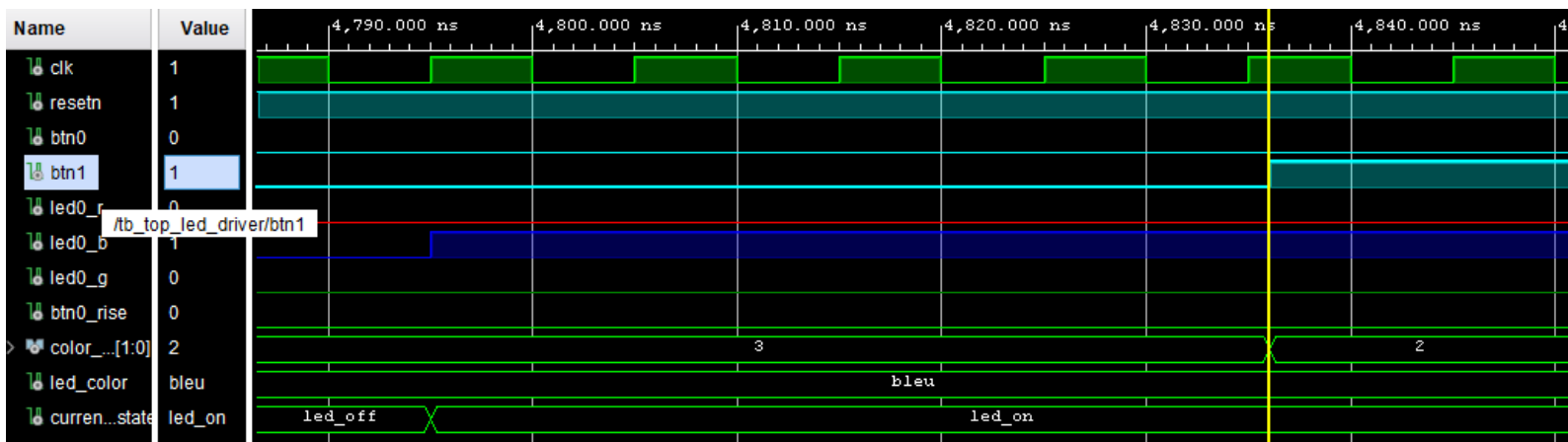
Lors de l'appui sur btn0:

- =>le front montant sur btn0 est détecté au front d'horloge suivant
- =>le signal led_color change sur le même front d'horloge, sa valeur est fonction de la valeur de color_code
ici color_code = 11 donc led_color = bleu
- =>les signaux de contrôle de la led basculent aussi sur le même front d'horloge
ici led0_g = 0 et led0_b =1

Note Le changement de couleur se fait sur une même periode d'allumage de la LED.

Le temps d'allumage de 200ns est partagé entre la couleur verte et le couleur bleue ($T_0 + T_1 = 200\text{ns}$)

- Validation de l'appui sur btn1



Lors de l'appui sur btn1, le color_code est mis à jour de manière asynchrone mais le changement sur l'affichage de la LED ne sera effectif qu'au prochain appui sur btn0.

L'appui sur btn1 n'a pas d'impact immédiat sur l'affichage de la LED

13. Réalisez une synthèse et étudiez le rapport de synthèse, les ressources utilisées doivent correspondre à votre schéma RTL.

- Définition du fichier de contraintes

```
# PL System Clock
set_property -dict { PACKAGE_PIN H16 IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L13P_T2_MRCC_35 Sch=sysclk
create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk }];#set

# RGB LEDs
set_property -dict { PACKAGE_PIN L15 IOSTANDARD LVCMOS33 } [get_ports { led0_b }]; #IO_L22N_T3_AD7N_35 Sch=led0_b
set_property -dict { PACKAGE_PIN G17 IOSTANDARD LVCMOS33 } [get_ports { led0_g }]; #IO_L16P_T2_35 Sch=led0_g
set_property -dict { PACKAGE_PIN N15 IOSTANDARD LVCMOS33 } [get_ports { led0_r }]; #IO_L21P_T3_DQS_AD14P_35 Sch=led0_r
#set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports { led1_b }]; #IO_0_35 Sch=led1_b
#set_property -dict { PACKAGE_PIN L14 IOSTANDARD LVCMOS33 } [get_ports { led1_g }]; #IO_L22P_T3_AD7P_35 Sch=led1_g
#set_property -dict { PACKAGE_PIN M15 IOSTANDARD LVCMOS33 } [get_ports { led1_r }]; #IO_L23N_T3_35 Sch=led1_r

# Buttons
set_property -dict { PACKAGE_PIN D20 IOSTANDARD LVCMOS33 } [get_ports { btn0 }]; #IO_L4N_T0_35 Sch=btn[0]
set_property -dict { PACKAGE_PIN D19 IOSTANDARD LVCMOS33 } [get_ports { btn1 }]; #IO_L4P_T0_35 Sch=btn[1]

## User Digital I/O Header J1
set_property -dict { PACKAGE_PIN L19 IOSTANDARD LVCMOS33 } [get_ports { resetn }]; #IO_L9P_T1_DQS_AD3P_35 Sch=user_dio[1]
#set_property -dict { PACKAGE_PIN M19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[2] }]; #IO_L7P_T1_AD2P_35 Sch=user_dio[2]
#set_property -dict { PACKAGE_PIN N20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[3] }]; #IO_L14P_T2_SRCC_34 Sch=user_dio[3]
#set_property -dict { PACKAGE_PIN P20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[4] }]; #IO_L14N_T2_SRCC_34 Sch=user_dio[4]
```

La fréquence de l'horloge sur la carte est de 125Mhz soit une période de 8ns.

le signal de resetn est routé sur un user_io[1] pour pouvoir être connecté a une alimentation sur la carte

- Résultat de la synthèse

Start RTL Component Statistics

Detailed RTL Component Info :

+---Adders :

```
2 Input      28 Bit      Adders := 1
```

1 additionneur du counter_unit sur 28 bits

+---Registers :

```
28 Bit    Registers := 1
```

1 registre 28 bits pour la cible du counter_unit

1 Bit Registers := 3

1 registre 1 bit pour la machine d'état onoff.

+---Muxes :

```
2 Input      28 Bit      Muxes := 1
```

2 registres 1 bit pour la gestion de front montant de btn0

```
4 Input      2 Bit      Muxes := 1
```

1 mux 28 bits pour le counter_unit

```
2 Input      2 Bit      Muxes := 1
```

1 mux 2 bits 4 entrées pour le décodeur de code couleur

```
2 Input      1 Bit      Muxes := 1
```

1 mux 2 bit 2 entrées pour le codage du code couleur

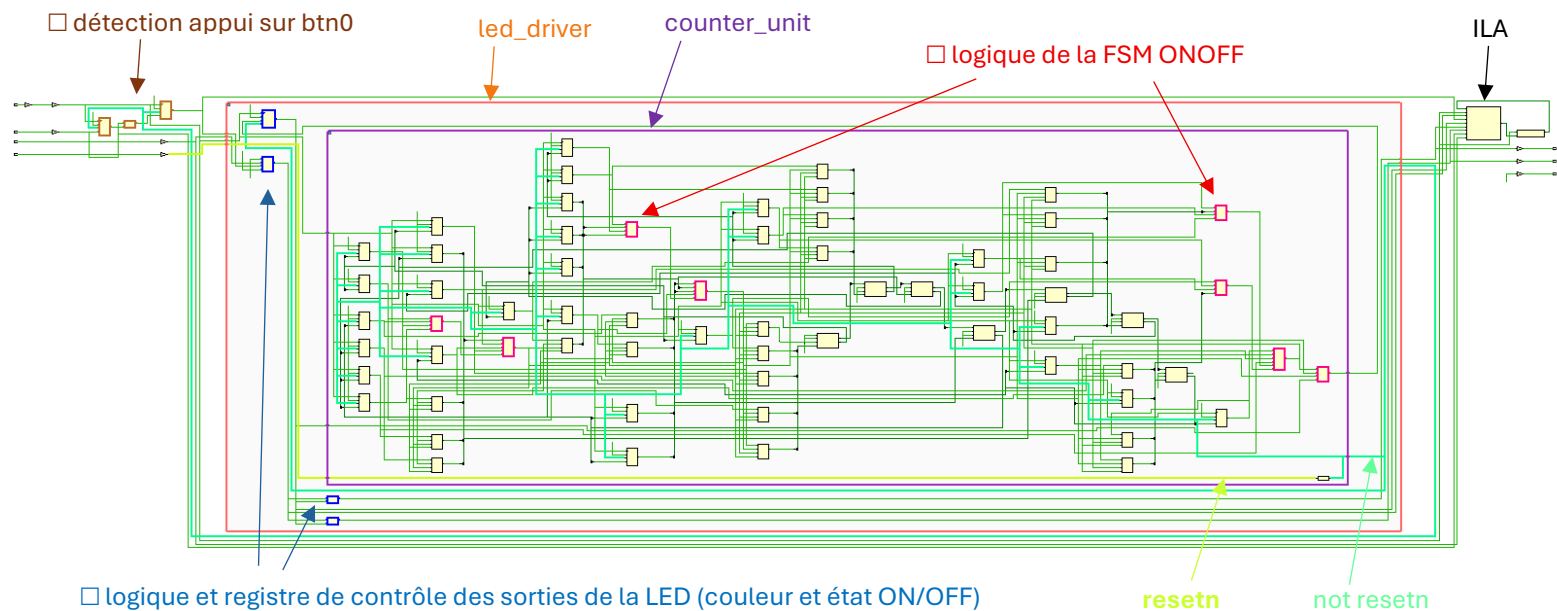
1 mux 1 bit 2 entrées pour la machine d'état onoff

Report Cell Usage:

	Cell	Count
1	BUFG	1
2	CARRY4	7
3	LUT1	1
4	LUT2	3
5	LUT4	26
6	LUT5	2
7	LUT6	1
8	FDCE	30
9	FDPE	1
10	LD	1
11	IBUF	4
12	OBUF	3

1 BUFG :	1 buffer d'horloge
7 CARRY4 :	7 additionneurs de 4 bits pour le compteur 28 bits
1 LUT 1	1 inverseur pour le resetn
3 LUT 2	1 LUT2 pour la logique de détection du front montant sur btn0 2 LUT2 pour la logique de contrôle des sorties led_g et led_b
26 LUT4:	21 LUT4 pour le compteur 28 bits 5 LUT pour la logique sur la machine d'état onoff
2 LUT5:	2 LUT5 pour la logique sur la machine d'état onoff
1 LUT6	2 LUT5 pour la logique sur la machine d'état onoff
30 FDCE	28 registres pour le compteur 2 registres pour la détection du front montant de btn0
1 FDPE	1 registre pour la machine d'état onoff qui vient contrôler la couleur de la LED
1 LD	1 registre pour gérer la couleur qui vient contrôler la couleur de la LED
4 IBUF	4 buffers d'entrée pour les 4 entrées clk, resetn, btn0 et btn 1
3 OBUF	3 buffers de sortie pour les 3 sorties de contrôle de la LED : led_r, led_g, led_b

• Identification des blocs sur le schéma implémenté



- Note 1 : la logique de contrôle du code couleur a été simplifié par :
 - o color_code[1] = 1
 - o et color_code[0] = btn1
- Note 2 : le signal resetn est inversé pour contrôler tous les

• Implémentation / rapport de timings

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1,489 ns	Worst Hold Slack (WHS): 0,023 ns	Worst Pulse Width Slack (WPWS): 2,750 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 4027	Total Number of Endpoints: 4011	Total Number of Endpoints: 2272
All user specified timing constraints are met.		

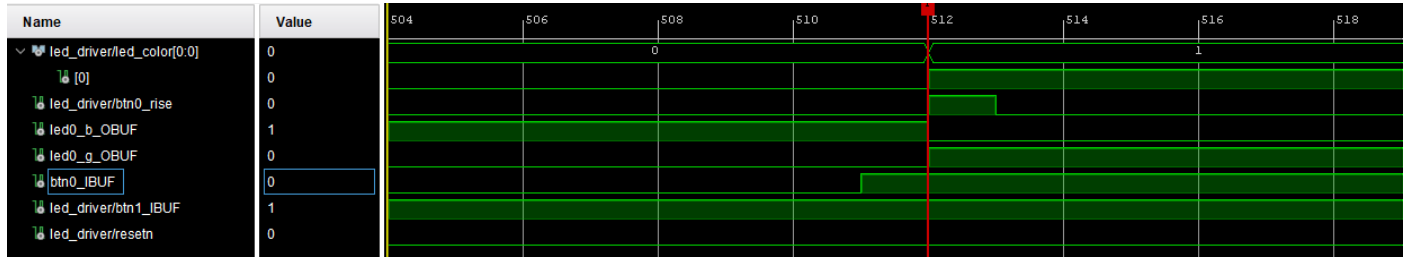
Le design respecte les contraintes de timings pour une horloge à 125Mhz.
 Il n’y a pas de violation de setup et de hold TNS = 0 et THS= 0.

- Validation sur la carte: mesures sur l'ILA

- a. Validation du changement de couleur bleu => vert sur l'ILA

Configuration : Le trigger est réglé sur front montant de btn0

Déroulé de la mesure : btn1 est appuyé, on appuie sur btn0 quand la LED est allumée en bleu



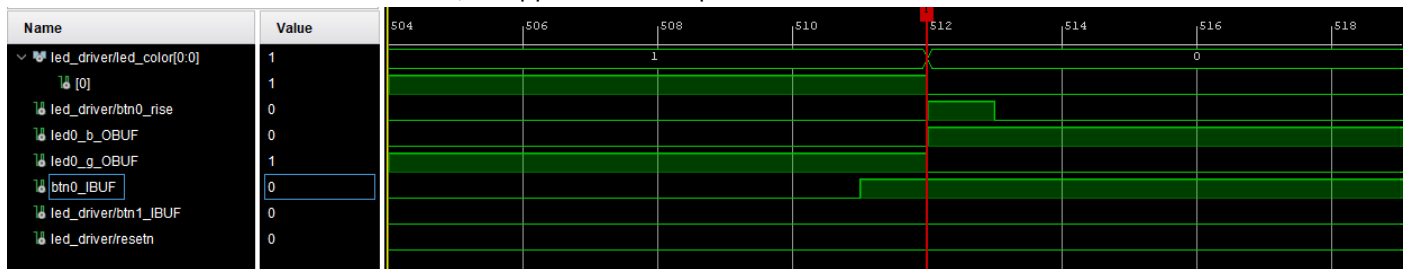
A l'appui sur btn0 :

- btn0 passe à 1
- la consigne de code couleur passe à 1 (on mesure uniquement de bit de poids faible du code couleur car c'est lui qui modifie la couleur entre bu et vert
- btn0_rise passe à 1 pendant une periode d'horloge
- led_b passe à 0
- led_g passe à 1: le LED clignote vert

- b. Validation du changement de couleur vert => bleu sur l'ILA

Configuration : Le trigger est réglé sur front montant de btn0

Déroulé de la mesure : btn1 est relâché, on appuie sur btn0 quand la LED est allumée en vert



A l'appui sur btn0 :

- btn0 passe à 1
- la consigne de code couleur passe à 0 sur le bit[0]
- btn0_rise passe à 1 pendant une periode d'horloge
- led_g passe à 0
- led_b passe à 1: le LED clignote bleu

- Validation sur la carte: mesures à l'oscilloscope

Afin de mesurer plus facilement sur l'oscilloscope:

Les sorties led_g et led_b sont dupliquées et routées sur des ports user_io.

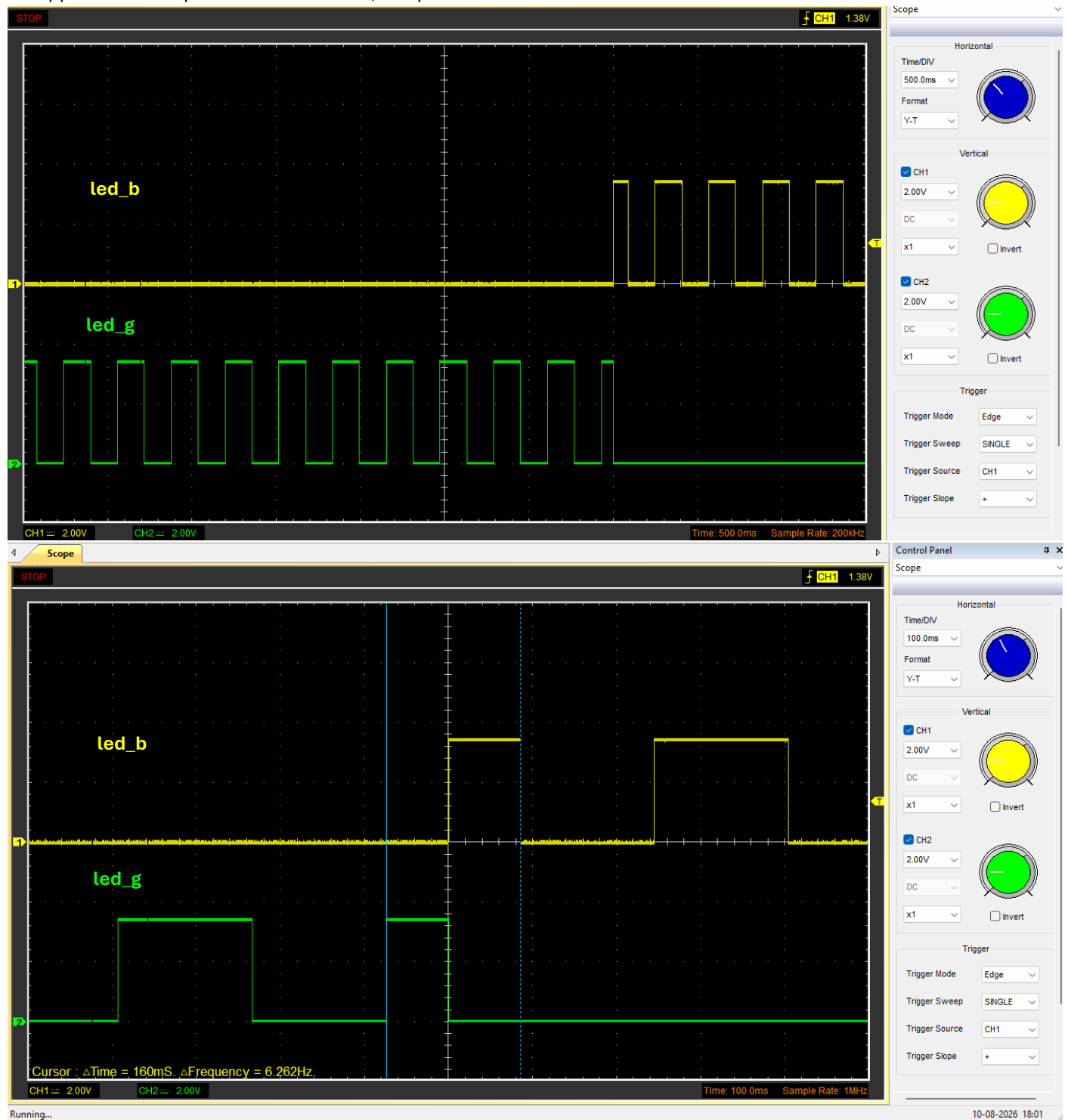
La période d'allumage est réduite à 0.2secondes, le cycle de comptage est donc réduit à 20 000 000

L'implémentation est refaite avec ses paramètres.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity top_led_driver is
  generic (
    cible : integer := 20000000
  );
  port (
    clk      : in std_logic;
    resetn   : in std_logic;
    btn0     : in std_logic;
    btn1     : in std_logic;
    led0_r    : out std_logic;
    led0_b    : out std_logic;
    led0_g    : out std_logic;
    test_led0_b : out std_logic;
    test_led0_g : out std_logic
  );
end top_led_driver;
architecture behavioral of top_led_driver is
  signal color_code : STD_LOGIC_VECTOR (1 downto 0) := "00";
  signal btn0_old : std_logic := '0';
  signal btn0_rise : std_logic := '0';
  signal in_led0_b : std_logic := '0';
  signal in_led0_g : std_logic := '0';
begin
  led_driver: entity work.led_driver(behavioral)
    generic map (
      cible => cible
    )
    port map (
      clk => clk,
      resetn => resetn,
      color_code => color_code,
      update => btn0_rise,
      led_r => led0_r,
      led_g => in_led0_g,
      led_b => in_led0_b
    );
  process(clk, resetn)
  begin
    if resetn = '0' then
      btn0_old <= '0';
      btn0_rise <= '0';
    elsif rising_edge(clk) then
      btn0_old <= btn0;
      if btn0_old = '0' and btn0 = '1' then
        btn0_rise <= '1';
      else
        btn0_rise <= '0';
      end if;
    end if;
  end process;
  led0_g <= in_led0_g;
  led0_b <= in_led0_b;
  test_led0_g <= in_led0_g;
  test_led0_b <= in_led0_b;
  color_code <= "10" when btn1 = '1' else "11";
end behavioral;
```

a. Validation du changement de couleur vert => bleu

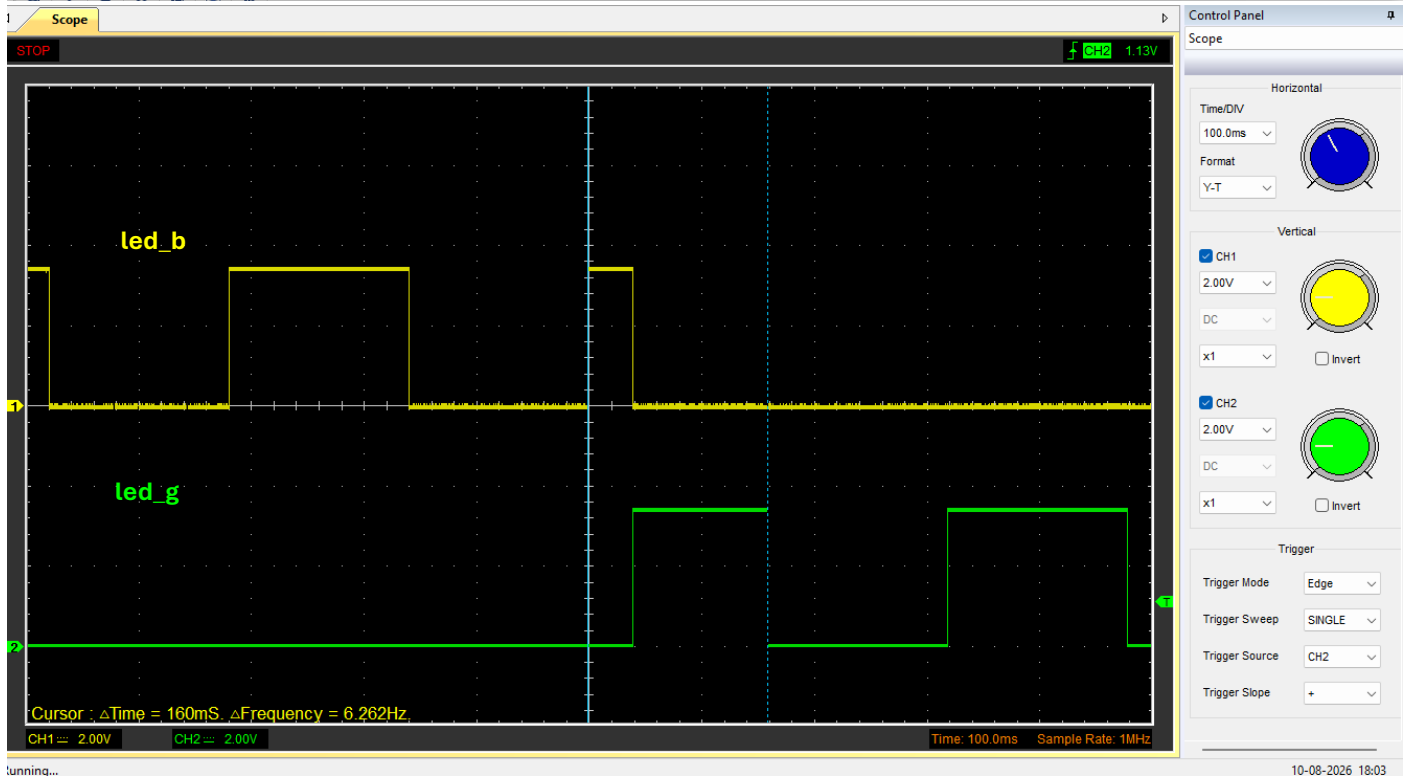
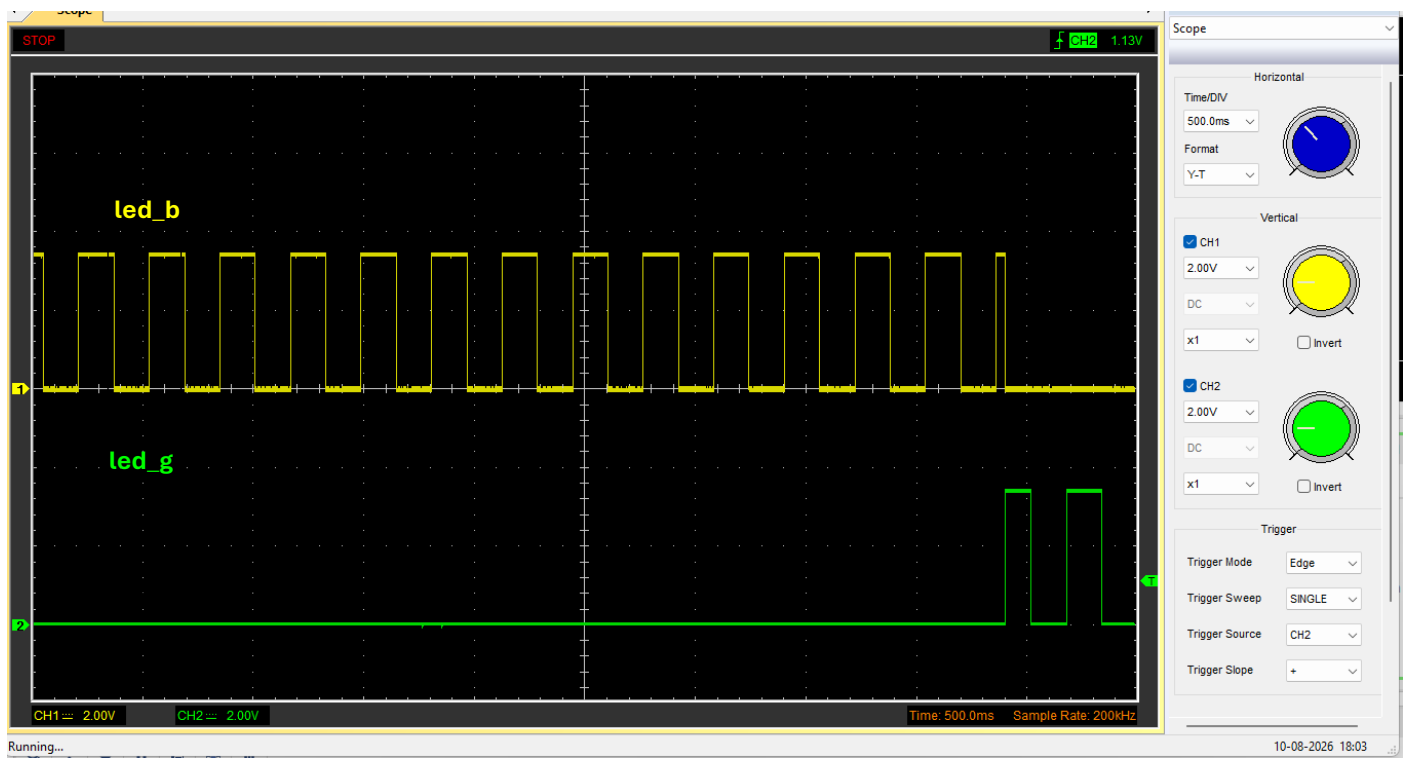
On appuie sur btn0 quand la LED est verte, elle passe immédiatement en bleu



La durée d'allumage est de 160ms ce qui correspond à $20 \cdot 10^6 / 125 \cdot 10^6 = 160\text{ms}$

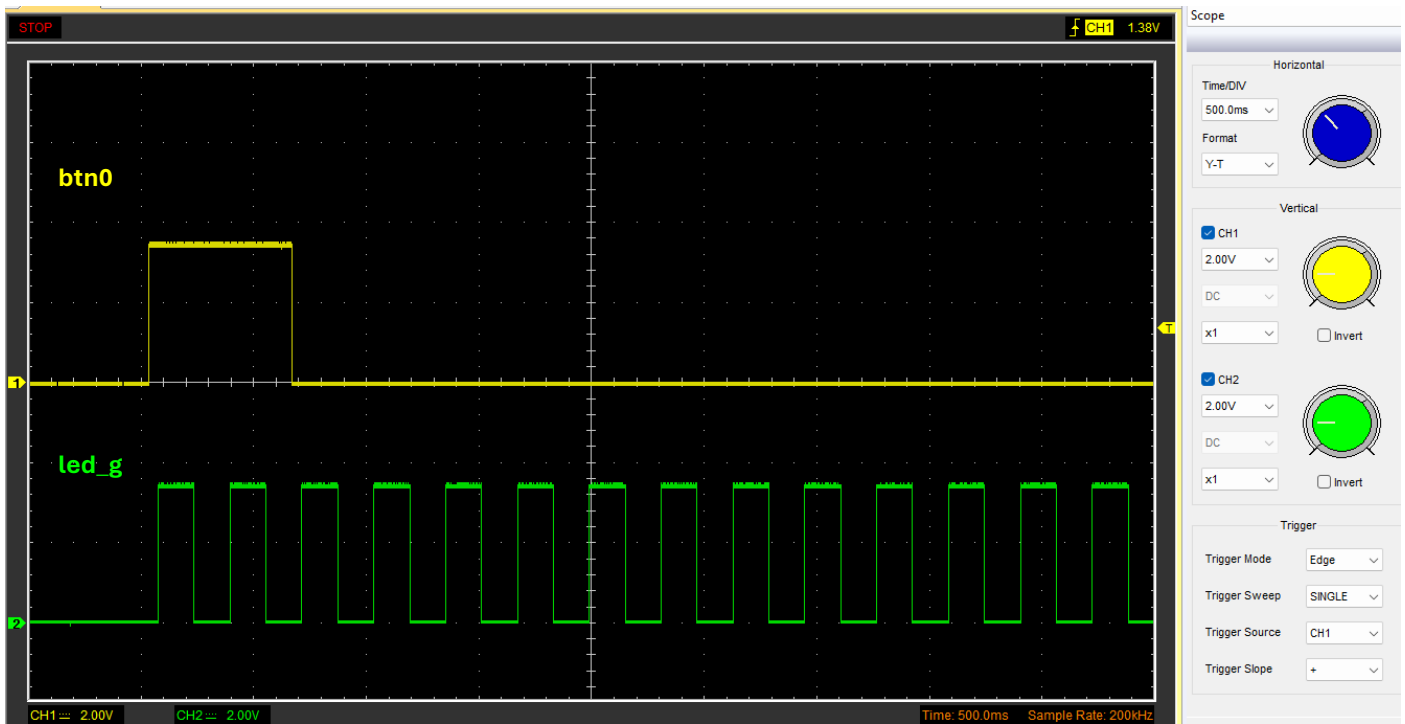
b. Validation du changement de couleur bleu => vert

On appuie sur btn 1 puis sur btn0 quand la LED est bleu, elle passe immédiatement en vert

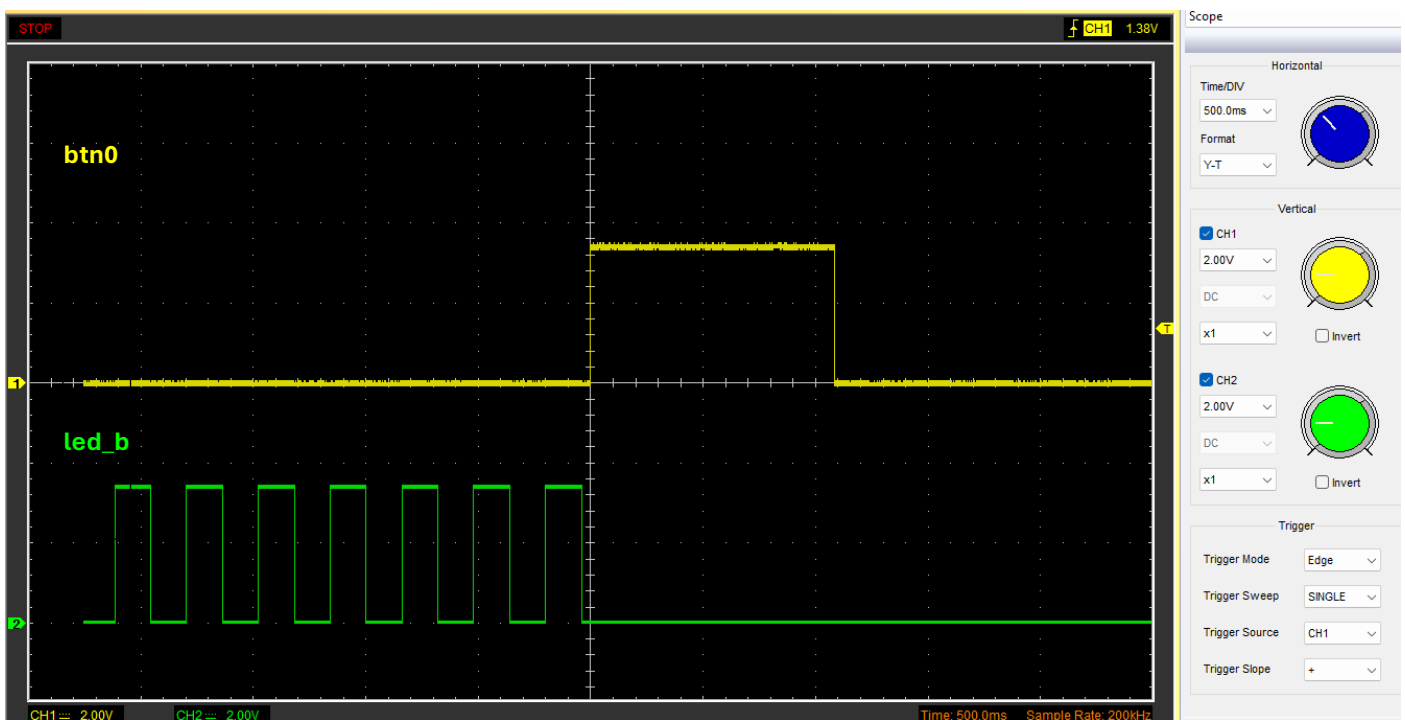


La durée d'allumage est de 160ms ce qui correspond à $20 \cdot 10^6 / 125 \cdot 10^6 = 160\text{ms}$

c. Validation de l'appui sur btn0

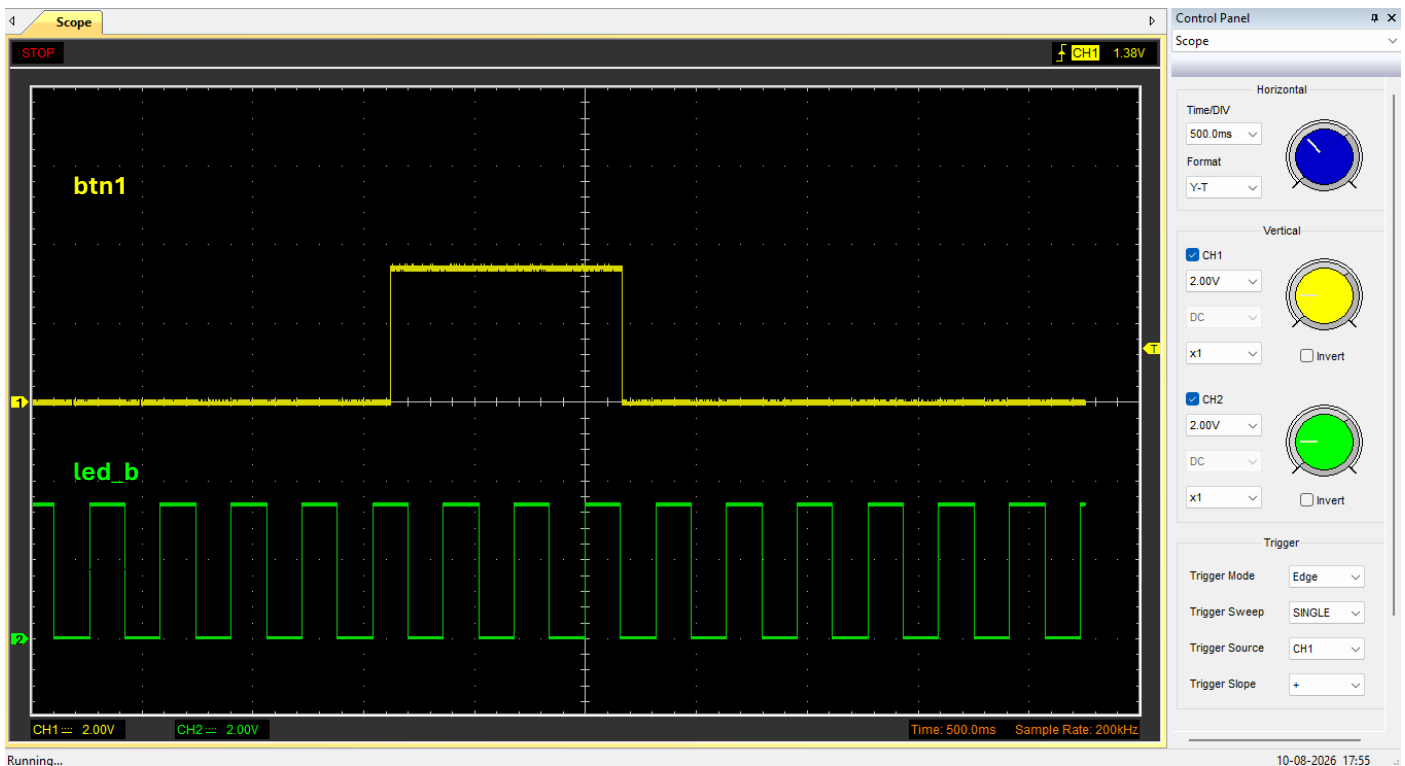


La LED passe au vert lors de l'appui sur btn0.

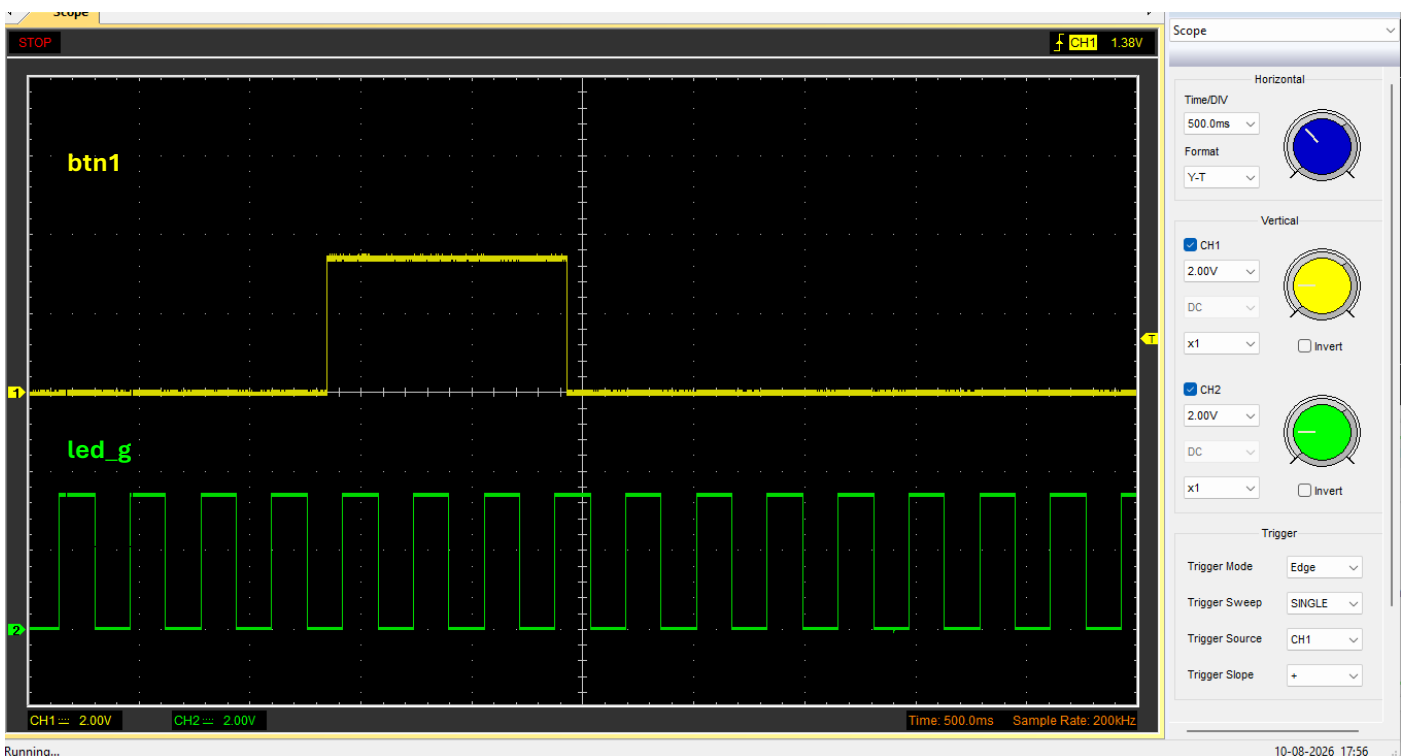


La couleur bleu s'éteint lors de l'appui sur btn0.

d. Validation de l'appui sur btn1



L'appui sur btn1 n'a aucun effet sur le changement de couleur



L'appui sur btn1 n'a aucun effet sur le changement de couleur

Conclusion

Le pilotage de la LED fonctionne comme prévu :

Lorsque le btn0 vient d'être appuyé, la couleur de la LED est mise à jour

Le maintien de btn0 à 1 n'a aucun effet, même si on appuis sur btn1

Si le btn1 est pressé la LED passe au vert au prochain appui sur btn0

Si le btn1 est relâché la LED passe au bleu au prochain appui sur btn0